

MM-ContextFold: Context Folding for Multimodal Agentic Retrieval

Yang Tian*
Shandong University
Jinan, China
tianyangchn@gmail.com

Fan Liu†
Southeast University
Nanjing, China
liufancs@gmail.com

Jingyuan Zhang
Kuaishou
Beijing, China
zhangjingyuan1994@gmail.com

Zhenyang Li
Hong Kong University of Science and
Technology
Hong Kong, Hong Kong
zhenyanglidz@gmail.com

Yupeng Hu
Shandong University
Jinan, China
huyupeng@sdu.edu.cn

Liqiang Nie†
Harbin Institute of Technology (Shenzhen)
Shenzhen, China
nieliqiang@gmail.com

Abstract

Multimodal Agentic Retrieval (MAR) requires agents to solve complex information-seeking tasks by iteratively invoking external tools. Typical frameworks such as ReAct maintain raw multimodal inputs and the accumulating interaction history in a single, ever-growing context, leading to the context explosion problem. While existing methods alleviate this issue by compressing redundant text, effective strategies for managing token-intensive visual content remain largely underexplored. To address this gap, we first conduct a systematic empirical study of approximately 10,000 trajectories. The results show that as visual cues are progressively extracted through external tools and textualized into the context, raw images become increasingly redundant. Continued image retention is associated with higher output entropy and can even degrade task accuracy. Motivated by these findings, we propose MM-ContextFold, a training-free framework that loads raw images only when needed. It maintains a persistent, text-only main context for high-level planning and spawns ephemeral branch contexts for image-dependent subtasks. Within each branch, the agent loads the relevant images, completes the subtask, and folds the result back into the main context as a concise textual summary; the images and branch trace are then discarded. Experiments on seven MAR benchmarks across five backbone models show that MM-ContextFold improves average accuracy by 6.3 percentage points over ReAct while reducing the working context length by 27.5%.

CCS Concepts

• **Information systems** → **Information retrieval**; **Question answering**; • **Computing methodologies** → **Intelligent agents**.

Keywords

Multimodal Search Agent; Information Retrieval; Question Answering; Context Management

*Work done during an internship at Kuaishou.

†Corresponding authors.



This work is licensed under a Creative Commons Attribution 4.0 International License.
MM '26, Rio de Janeiro, Brazil

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2213-4/2026/11
<https://doi.org/10.1145/3767308.3836039>

Tool Call Analysis per Step

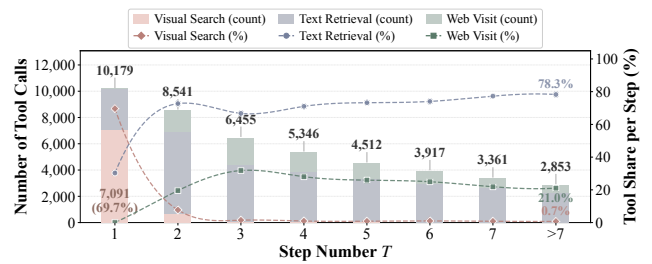


Figure 1: Per-step tool-call counts (bars) and proportions (lines) for *Visual Search*, *Text Retrieval*, and *Web Visit*.

ACM Reference Format:

Yang Tian, Fan Liu, Jingyuan Zhang, Zhenyang Li, Yupeng Hu, and Liqiang Nie. 2026. MM-ContextFold: Context Folding for Multimodal Agentic Retrieval. In *Proceedings of the 34th ACM International Conference on Multimedia (MM '26)*, November 10–14, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 20 pages. <https://doi.org/10.1145/3767308.3836039>

1 Introduction

Multimodal Agentic Retrieval (MAR) [8, 23, 38, 43] has recently emerged as a challenging task setting in deep research [5, 15, 24, 25]. Such tasks typically involve identifying long-tail entities and scenarios depicted in query images, as well as gathering time-sensitive or fine-grained information about them. These demands often lie beyond the reach of a model’s parametric knowledge or a single retrieval step. MAR agents therefore build on reasoning frameworks such as ReAct [1, 5, 35, 38] to plan and iteratively invoke external tools, retaining the multimodal input while appending each step’s reasoning text, tool calls, and observations to a single, ever-growing context. In long-horizon MAR tasks, this unconstrained accumulation of multimodal information rapidly leads to the context explosion problem, where redundant retrieval results and trial-and-error traces overwhelm the model’s reasoning capacity [22, 31, 36].

We further investigate this problem by examining which tools introduce external evidence at successive retrieval steps. Specifically, we collect approximately 10,000 trajectories and analyze the per-step use of three external tools: *Visual Search*, *Text Retrieval*, and *Web Visit*. As shown in Figure 1, evidence acquisition exhibits a stage-wise pattern. *Visual Search* calls, through which the agent grounds the visual content of the query images, are concentrated

in the early steps; thereafter, evidence acquisition is dominated by *Text Retrieval* and *Web Visit*. While the resulting redundant textual observations can be compressed by existing methods [22, 31, 36], no comparable mechanism governs the retention of raw images: these token-intensive inputs persist at every reasoning step even when no longer directly needed, continuously exacerbating the context explosion problem [32]. This naturally raises the question: *are raw images still necessary throughout the entire retrieval trajectory?*

To answer this question, we conduct a systematic empirical study that goes beyond tool-call statistics (§3.2). Specifically, we classify each reasoning step by whether it draws directly on raw images, relies on previously textualized visual evidence, or is independent of the images; probe, via output entropy, whether images still implicitly influence the model’s output distribution; and measure how removing images from a given step onward affects task accuracy (Figure 2). Together, these analyses show that the agent’s reliance on raw images is concentrated in an early grounding phase, during which task-relevant visual cues are progressively textualized into the context. Beyond this phase, retaining images increases output uncertainty and can degrade task accuracy [9, 26]. However, the duration of this phase varies with each task’s visual complexity, making any fixed removal step unlikely to generalize across tasks.

Motivated by these findings, we propose **MM-ContextFold**, a training-free context management framework that enables on-demand loading and timely release of raw images within the agent’s working context. MM-ContextFold introduces a dual-state mechanism that separates high-level planning from visual grounding: the agent alternates between a text-only *main state* and an ephemeral multimodal *branch state*. In the main state, the agent reasons over a persistent main context that maintains task progress, the overall strategy, and distilled evidence. When new visual cues are required, the agent initiates an image-dependent subtask and transitions to the branch state, where it loads the relevant raw images into a scoped branch context and executes the subtask through interleaved reasoning and tool calls. Upon completing the subtask, the agent folds its findings back into the main context as a concise textual summary; the raw images and the local branch trace are then discarded. By confining raw images to bounded branch scopes, MM-ContextFold allows textualized evidence to continuously enrich the main context, enabling more informed subsequent planning, while avoiding the accumulation of visual tokens. Experiments on seven MAR benchmarks across five backbone models show that MM-ContextFold improves average accuracy by 6.3 percentage points over the standard ReAct baseline while reducing working context length by 27.5%. Code and data are released at <https://github.com/iLearn-Lab/MM26-MMContextFold>. In summary, our main contributions are as follows:

- We conduct an in-depth empirical study of approximately 10,000 multimodal retrieval trajectories, providing, to our knowledge, the first systematic empirical characterization of the dynamically diminishing pattern of agent reliance on images in MAR, and demonstrating that retaining images after visual cues have been textualized can degrade reasoning accuracy.
- Based on this insight, we propose MM-ContextFold, a dual-state context architecture that structurally decouples visual-grounding steps from the main reasoning trajectory, achieving on-demand loading and timely release of raw images.

- Extensive experiments on seven MAR benchmarks across five backbone models show that MM-ContextFold reduces working context length by 27.5% while improving average accuracy by 6.3 percentage points over the standard ReAct baseline.

2 Related Work

Multimodal Agentic Retrieval. LLM-based agents have demonstrated strong capabilities in autonomous information seeking, evolving from text-only deep research systems [16, 29, 30, 41] to multimodal agentic retrieval (MAR), where queries involve images alongside text [8, 23, 34, 38, 43]. Since neither the model’s parametric knowledge nor a single retrieval step suffices for such queries, MAR requires agents to iteratively invoke external search tools to gather, verify, and integrate multimodal evidence. This iterative process distinguishes MAR from multimodal RAG pipelines, which operate over closed or pre-collected corpora without agentic tool interaction [28, 40]. To evaluate MAR, several benchmarks have been proposed [6, 7, 10, 38]. On the system side, the ReAct framework [35], already a mainstream architecture for text-only deep research agents, has been further adopted as the backbone of MAR systems. Recent efforts [4, 8, 39] mainly focus on improving agent capabilities through reinforcement learning (RL). While these training-centric efforts advance MAR, they retain the append-only context mechanism inherited from ReAct. Under this mechanism, the context explosion problem observed in text-only agents [31, 36] is further amplified by the persistent accumulation of token-intensive visual content alongside textual interaction traces.

Context Management for Search Agents. Context management for LLM agents broadly falls into two categories: *external context augmentation*, which injects knowledge from outside the current trajectory, such as user profiles or past conversations [3, 17, 33], and *intra-task context curation*, which manages the context generated within the task itself over long horizons [19, 20]. Early intra-task methods adopt stepwise summarization that compresses the full history at each turn [31, 37, 44], but such uniform compression risks discarding fine-grained details critical for complex tasks. Two representative works move beyond this limitation by introducing *agentic context folding*. AgentFold [36] treats context as a dynamic cognitive workspace, enabling the agent to selectively fold multi-step interactions at different scales, from condensing individual steps to consolidating completed sub-investigations, and maintains only 7K tokens after 100 interaction turns. ContextFold [22] proposes a branch-and-return mechanism with FoldGRPO, an end-to-end RL algorithm that trains the agent’s branching behavior through dense process rewards to achieve high context compression. Despite this progress, existing methods mainly target text-only scenarios. When extended to multimodal retrieval, they face a structural mismatch: raw images are needed only *locally* along the trajectory, and their grounding results can be preserved as compact text, yet uniformly compressing or persistently retaining the images either loses fine-grained visual details or inflates the context and degrades accuracy. This mismatch motivates our work.

3 MM-ContextFold

This section first formalizes the MAR task and its standard agentic framework (§3.1), then presents an empirical study on the role

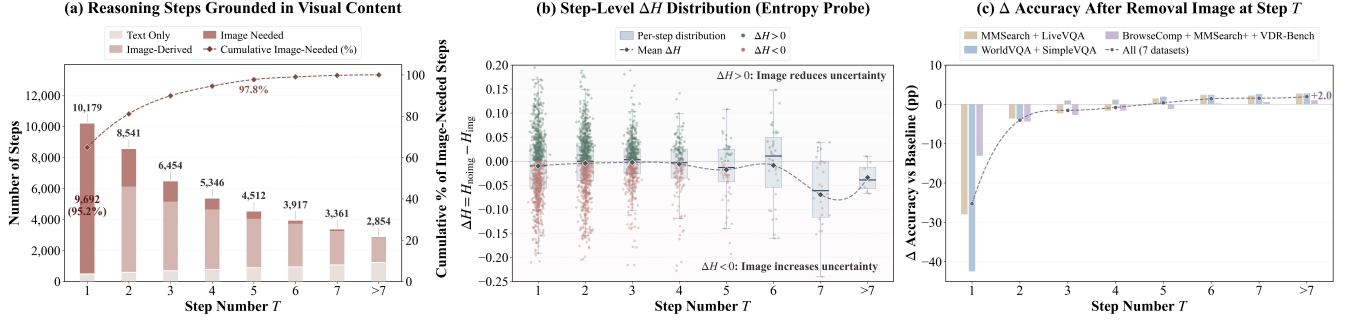


Figure 2: Empirical analysis on the role of images in MAR trajectories. (a) Per-step breakdown of reasoning texts into *Image Needed* (grounded in new visual information directly extracted from the raw images), *Image-Derived* (based on visual information already textualized in earlier steps), and *Text Only*; the dashed line shows the cumulative proportion of *Image Needed* instances. (b) Step-level entropy change induced by image removal, measured as $\Delta H_t = H(\tau_t^{\text{IR}}) - H(\tau_t)$; box plots show the per-step distributions and the dashed line connects the step-level means. (c) $\Delta\text{Accuracy}$ (%) relative to the with-image baseline when images are removed from step T onward; grouped bars show the three benchmark categories, and the dashed line shows the weighted average over all seven benchmarks.

of raw images across retrieval trajectories (§3.2), whose findings motivate the design of the MM-ContextFold framework (§3.3).

3.1 Task Formulation

We formalize MAR as a sequential decision-making process. Given a multimodal input $(q, \mathcal{I})$, where q denotes a natural-language question and $\mathcal{I} = \{I_k\}_{k=1}^K$ denotes a set of query images, the agent $\mathcal{M}_\theta$ produces an answer a through multi-step interaction with an external environment. At each step t , conditioned on the accumulated context τ_t , the agent generates a reasoning text r_t and invokes a tool $u_t \in \mathcal{U}$; the returned observation o_t , together with (r_t, u_t) , is then appended to the context to form τ_{t+1} :

$$\begin{cases} \tau_t := (q, \mathcal{I}, r_1, u_1, o_1, \dots, r_{t-1}, u_{t-1}, o_{t-1}), \\ (r_t, u_t) \leftarrow \mathcal{M}_\theta(\tau_t). \end{cases} \quad (1)$$

In the setting examined in this paper, the tool set $\mathcal{U}$ comprises three tool types: *Visual Search*, which retrieves visual references and source links based on $\mathcal{I}$; *Text Retrieval*, which performs open-domain retrieval and returns relevant webpage metadata and links; and *Web Visit*, which accesses a specified URL and extracts the parsed content of the webpage.

3.2 Empirical Analysis

As shown in Figure 1, the tool-call distribution of approximately 10,000 ReAct trajectories¹ reveals that *Visual Search* is concentrated in the first two steps, after which the agent shifts almost exclusively to *Text Retrieval* and *Web Visit*. This behavioral pattern provides preliminary evidence that the agent’s reliance on raw images is confined to the early stages. However, tool-call distributions characterize visual reliance only in the action component u_t ; they cannot reveal whether the reasoning component r_t remains grounded in

raw image content, whether images implicitly influence the model’s output distribution, or how their removal affects final task accuracy. We therefore analyze the role of raw images from three complementary angles: the visual grounding of reasoning texts (§3.2.1), a stepwise entropy probe on output distributions (§3.2.2), and step-level image-removal ablations on task accuracy (§3.2.3).

3.2.1 Visual Grounding of Reasoning Text. To characterize how the reasoning text r_t at each step is grounded in the visual input, we employ Gemini-3-Flash as an annotator and label every reasoning step in the collected trajectories.² For each step t , the annotator receives the preceding context τ_t defined in Eq. (1) together with r_t , which it assigns to one of three mutually exclusive categories: (1) *Image Needed*, where r_t introduces visual information drawn directly from the query images $\mathcal{I}$ that has not appeared earlier in τ_t ; (2) *Image-Derived*, where r_t relies on visual information already textualized in earlier steps rather than on the raw images themselves; and (3) *Text Only*, where r_t draws on neither source of visual information. Figure 2(a) shows that 95.2% of Step-1 reasoning texts are labeled *Image Needed*, but this share falls sharply thereafter, and *Image-Derived* becomes the dominant category from Step 2 onward, while *Text Only* rises steadily. Overall, 97.8% of all steps labeled *Image Needed* occur within the first five steps. Together with the tool-call distribution in Figure 1, these results indicate that the agent’s direct use of raw images is restricted to an early visual-grounding phase, during which task-relevant visual information is extracted and converted into textual artifacts.

3.2.2 Entropy Probe. The analysis of output text (r_t, u_t) shows that raw images are not uniformly necessary throughout the retrieval trajectory. However, this conclusion does not rule out the possibility that raw images still implicitly influence the model’s output distribution at later steps. To test this, we introduce a stepwise output-entropy probe [11, 12, 18]. Specifically, let τ_t denote the original multimodal context at step t as defined in Eq. (1), and τ_t^{IR} denote the context obtained by removing the query images $\mathcal{I}$ from

¹Cross-model trajectories span Gemini-3-Flash, GPT-5.2, Qwen3.5-9B, Qwen3.5-27B, and Qwen3.5-35B-A3B. Trajectories are collected via both API services and local deployment; of the roughly 10,000 collected trajectories, 5,087 valid ones are from locally deployed Qwen-series models. Because token-level probabilities are available only from local deployment, we conduct the entropy probe on this locally deployed Qwen subset; for consistency, the paired step-level image-removal ablations are also run on the same subset.

²We validate annotation quality on a random subsample of 500 steps independently labeled by three human annotators; the LLM annotations achieve a 93.5% agreement rate with the majority human vote.

τ_t . For the reasoning text $r_t = (y_1^{(t)}, \dots, y_{m_t}^{(t)})$, where $y_i^{(t)}$ denotes the i -th token and m_t the sequence length, its length-normalized entropy under context τ_t is:

$$\mathbf{H}(\tau_t) = -\frac{1}{m_t} \sum_{i=1}^{m_t} \sum_{v \in \mathcal{V}} p(v | \tau_t, y_{<i}^{(t)}) \log p(v | \tau_t, y_{<i}^{(t)}), \quad (2)$$

where $\mathcal{V}$ denotes the vocabulary and $p(v | \tau_t, y_{<i}^{(t)})$ is the next-token probability assigned by the model $\mathcal{M}_\theta$. We further define the entropy change induced by image removal:

$$\Delta H_t = \mathbf{H}(\tau_t^{\text{IR}}) - \mathbf{H}(\tau_t). \quad (3)$$

Negative ΔH_t indicates that retaining images increases output entropy. As shown in Figure 2(b), ΔH_t exhibits a clear stepwise pattern: at Steps 1–2, ΔH_t is slightly negative, suggesting that without visual input the model falls back on language priors [13, 27, 42], yielding more confident yet less visually informed predictions; during Steps 3–6, $|\Delta H_t|$ remains near zero, suggesting that the output distribution is largely unaffected by image removal; and from Step 7 onward, ΔH_t shifts to more negative values, indicating that beyond the visual-grounding steps, raw images may introduce redundant context into the predominantly textual reasoning [14, 21].

3.2.3 Image-Removal Ablation. We further conduct ablations using τ_t^{IR} , i.e., the context with query images removed, to measure the direct impact on task accuracy. Starting from step t , the model continues the standard ReAct loop based on τ_t^{IR} until termination. Let T_i denote the total number of steps in the original trajectory of sample i , and $\mathcal{X}_t = \{i \mid T_i \geq t\}$ the set of samples whose original trajectories reach step t . Let $\hat{a}_i^{\text{IR},t}$ and $\hat{a}_i$ denote the final answers of the image-removed and original runs, respectively. We report the accuracy difference over $\mathcal{X}_t$:

$$\Delta \text{Acc}(t) = \frac{1}{|\mathcal{X}_t|} \sum_{i \in \mathcal{X}_t} \left(\mathbf{1}[\hat{a}_i^{\text{IR},t} = a_i^*] - \mathbf{1}[\hat{a}_i = a_i^*] \right). \quad (4)$$

As shown in Figure 2(c), removing images during the first two steps causes the most severe accuracy degradation (−25.2% at Step 1; −3.9% at Step 2), aligning with the output-text analysis in §3.2.1 where *Visual Search* calls and raw-image-grounded reasoning are both concentrated. As the trajectory progresses, the impact of image removal diminishes steadily: the accuracy change approaches near zero by Step 4, and from Step 6 onward removing images consistently improves accuracy by +1% to +2% on average. We further disaggregate these results by three benchmark categories (§4.1). A clear contrast emerges at Step 5: whereas *Atomic factuality* (WorldVQA [43] and SimpleVQA [2]) and *Dynamic information-seeking* benchmarks (MMSearch [10] and LiveVQA [7]) already yield positive gains, *Visual deep research* benchmarks (BrowseComp-VL [8], MMSearch-Plus [23], and VDR-Bench [38]), which involve more complex queries and longer retrieval trajectories, still show a slightly negative ΔAcc . This divergence across benchmarks suggests that although removing images in later steps can improve accuracy, the optimal removal step varies by task, making it difficult to define a single removal point that generalizes across benchmarks.

3.2.4 Key Findings. The three analyses above converge on a picture of how raw images contribute to MAR trajectories: (1) the agent’s reliance on raw images is concentrated in specific visual-grounding

steps; once the task-relevant visual cues are extracted and textualized, subsequent reasoning operates predominantly over textual artifacts (Figure 1, Figure 2(a)); (2) beyond the grounding phase, persistently retaining raw images provides little benefit (Figure 2(c)); instead, it introduces redundant visual context that increases output entropy, suggesting higher reasoning uncertainty (Figure 2(b)); and (3) the number of steps for which raw images remain beneficial varies across benchmark categories: *Atomic factuality* and *Dynamic information-seeking* tasks complete visual grounding earlier, whereas complex *Visual deep research* tasks require sustained image access over more steps (Figure 2(c)).

3.3 MM-ContextFold Framework

The empirical findings motivate three design principles: (P1) *decoupled reasoning and grounding*, separating textual planning and evidence synthesis from visual grounding to reflect their distinct cognitive roles; (P2) *ephemeral visual context*, confining raw images to scoped contexts rather than retaining them persistently, since their presence can complicate later text-dominant reasoning; and (P3) *adaptive image allocation*, determining when and which images to load to accommodate varying visual demands across tasks. To instantiate these principles, MM-ContextFold introduces a dual-state mechanism (Figure 3): the agent alternates between a text-only **main state** for planning and an ephemeral multimodal **branch state** for scoped grounding (P1); each branch loads only the designated images (P3), and upon completion, its findings are folded back as text and the branch context is discarded (P2).

3.3.1 Dual-State Mechanism. Given a natural language question q and a set of query images $\mathcal{I} = \{I_k\}_{k=1}^K$, we first assign each image a stable textual identifier α_k and construct an image registry:

$$\mathcal{P} = \{(\alpha_k, I_k)\}_{k=1}^K. \quad (5)$$

In the main state, the agent $\mathcal{M}_\theta$ starts from the initial main context $\tau_1 := (q, \mathcal{A})$, where $\mathcal{A} = \{\alpha_k\}_{k=1}^K$ is the set of image identifiers. The transitions between the two states are governed by two actions:

- $u_t^B = \text{BRANCH}(s_t, \mathcal{A}_t)$: Initiates a subtask s_t that requires visual information from a subset of images $\mathcal{A}_t \subseteq \mathcal{A}$ (possibly empty), triggering a transition to the branch state.
- $u_t^R = \text{RETURN}(e_t)$: Concludes the current subtask. The agent folds the gathered multimodal evidence into a structured textual summary e_t and returns to the main state.

A complete cycle consists of the agent generating a reasoning step r_t and a BRANCH action in the main state, executing the subtask in the branch state, and concluding with a RETURN action. Subsequently, the ephemeral branch trajectory is discarded, and the persistent main context τ_t is updated with the synthesized result:

$$\tau_{t+1} = (\tau_t, r_t, u_t^B, e_t). \quad (6)$$

The main and branch policies share the same backbone model $\mathcal{M}_\theta$ but operate under different action spaces: the main state emits only high-level control actions (BRANCH or FINISH), whereas a branch state either invokes a tool from $\mathcal{U}$ or terminates with RETURN.

3.3.2 Bootstrapping Image-Aware Planning. To support **adaptive image allocation (P3)**, the agent needs to dynamically decide which images are relevant for subsequent subtasks. However, effective planning requires a basic semantic understanding of the

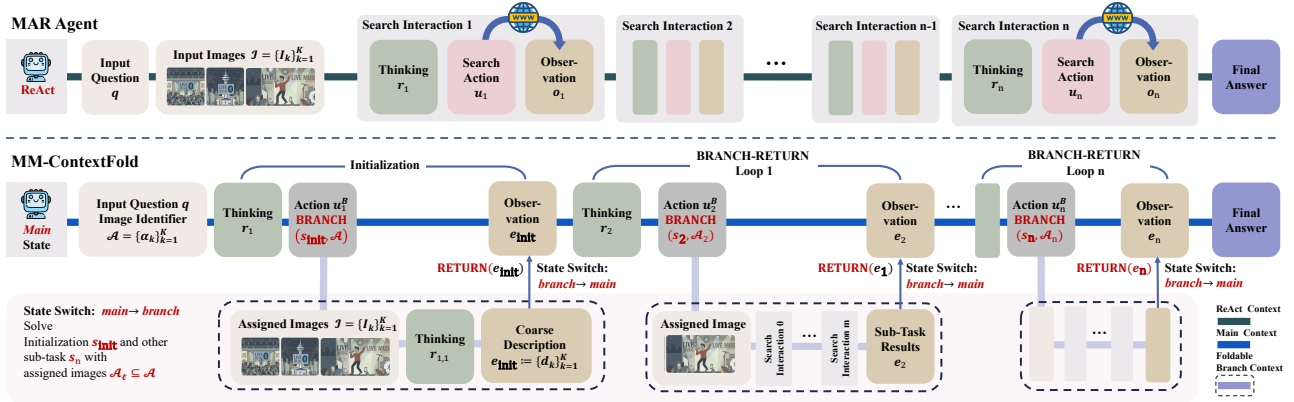


Figure 3: Overview of MM-ContextFold. *Top:* the standard MAR framework, where raw query images $\mathcal{I}$ persist in the context throughout the entire trajectory. *Bottom:* MM-ContextFold maintains a text-only *main state* that plans subtasks using coarse descriptions from initialization; the agent injects only the assigned images into an ephemeral *branch state* for visual grounding and tool use, and RETURN folds the branch result back as a textual summary, after which the branch context is discarded.

visual inputs that raw identifiers alone cannot provide. We therefore introduce an initialization cycle ($t = 1$) before the regular BRANCH-RETURN loop. From the initial main context τ_1 , the agent first issues $u_1^B = \text{BRANCH}(s_{\text{init}}, \mathcal{A})$:

$$(r_1, u_1^B) \leftarrow \mathcal{M}_\theta(\tau_1), \quad (7)$$

where s_{init} requests high-level overviews of the visual inputs. Upon completing s_{init} , the agent invokes $\hat{u}_{1,1}^R = \text{RETURN}(e_{\text{init}})$:

$$(\hat{r}_{1,1}, \hat{u}_{1,1}^R) \leftarrow \mathcal{M}_\theta(\hat{\tau}_{1,1}), \quad \hat{\tau}_{1,1} := (\tau_1, r_1, u_1^B, \mathcal{A}, \mathcal{I}). \quad (8)$$

The returned result $e_{\text{init}} := \{d_k\}_{k=1}^K$ consists of a textual description d_k for each image I_k . These descriptions are intentionally concise, capturing only high-confidence visual content to serve as reliable planning priors rather than detailed substitutes for raw images. Folding e_{init} back into the main state via Eq. (6) yields:

$$\tau_2 = (\tau_1, r_1, u_1^B, e_{\text{init}}). \quad (9)$$

3.3.3 Iterative Multimodal Branch-and-Return Reasoning. Equipped with τ_2 , the agent proceeds to the regular iterative loop ($t \geq 2$). Following the principle of decoupled reasoning and grounding (P1), the main state operates over textual artifacts. This separation allows the agent to focus on global task planning and information synthesis without reprocessing raw pixels. Given the current main context τ_t , the agent evaluates whether the accumulated information suffices to answer q . If so, it outputs the final answer; otherwise, it plans a new subtask s_t targeting the current information gap:

$$(r_t, u_t) \leftarrow \mathcal{M}_\theta(\tau_t), \quad u_t \in \{\text{FINISH}(a), \text{BRANCH}(s_t, \mathcal{A}_t)\}. \quad (10)$$

Here, $\mathcal{A}_t \subseteq \mathcal{A}$ dictates the specific images to be injected into the subsequent branch. $\mathcal{A}_t$ can be empty ($\emptyset$) if the required visual grounding has already been completed in prior iterations, enabling on-demand visual access. For a BRANCH action, the branch context is initialized by inheriting the main context τ_t alongside the on-demand raw images $\mathcal{I}_t$:

$$\hat{\tau}_{t,1} := (\tau_t, r_t, u_t^B, \mathcal{A}_t, \mathcal{I}_t), \quad \mathcal{I}_t = \{I \mid (\alpha, I) \in \mathcal{P}, \alpha \in \mathcal{A}_t\}. \quad (11)$$

This allows the agent to leverage global task background and prior folded results. Inside the branch, the same backbone model rolls

out a local policy over the branch context:

$$(\hat{r}_{t,j}, \hat{u}_{t,j}) \leftarrow \mathcal{M}_\theta(\hat{\tau}_{t,j}), \quad \hat{u}_{t,j} \in \mathcal{U} \cup \{\text{RETURN}(e)\}. \quad (12)$$

If $\hat{u}_{t,j} \in \mathcal{U}$, the branch executes the selected tool, receives observation $\hat{o}_{t,j}$, and appends $(\hat{r}_{t,j}, \hat{u}_{t,j}, \hat{o}_{t,j})$ to the local context to form $\hat{\tau}_{t,j+1}$. If $\hat{u}_{t,j} = \text{RETURN}(e_t)$, the branch terminates and sends the textual summary e_t back to the main state. The local trajectory is

$$\hat{\tau}_t := (\hat{\tau}_{t,1}, \hat{r}_{t,1}, \hat{u}_{t,1}, \hat{o}_{t,1}, \dots, \hat{r}_{t,J_t}, \hat{u}_{t,J_t}), \quad J_t \leq B_{\text{max}}, \quad (13)$$

where the final action $\hat{u}_{t,J_t}$ is always $\text{RETURN}(e_t)$. If the step budget is exhausted before an explicit RETURN, the final branch call is constrained to summarize the partial branch trajectory and terminate. The main context is subsequently updated according to Eq. (6). If the subtask is not fully resolved within the current cycle, the updated main context seamlessly carries over the partial progress, allowing the agent to allocate the same or different images in subsequent iterations to continue the grounding process.

3.3.4 Discussion. MM-ContextFold can be viewed as a dynamic context management framework that rethinks how visual information is integrated into long-horizon agentic reasoning. Rather than persistently retaining raw images or replacing them entirely with coarse textual surrogates, it operationalizes the three design principles to resolve the tension between reasoning efficiency and perceptual fidelity. The framework preserves direct access to original visual evidence within temporary branch contexts, folding the extracted findings back into the persistent textual trajectory. Unlike in generic context folding, the persistent and temporary states are modality-asymmetric rather than simply two scopes of the same mixed-modality history: the main state remains text-only, while original visual evidence is preserved only inside temporary branch contexts and folded back as text after use.

4 Experiments

4.1 Experimental Settings

Benchmarks. We evaluate MM-ContextFold on seven multimodal agentic retrieval benchmarks, grouped into three categories by their requirements for external knowledge retrieval and reasoning

Table 1: Main results on seven multimodal agentic retrieval benchmarks (accuracy, %). Subscripts give the number of evaluated instances per benchmark, identical for all methods and trials. All methods use the same tool set $\mathcal{U}$ and the same inference budget of up to 20 LLM calls. Within each backbone block, the best results are bold and the second-best underlined. Avg. is weighted by these counts. Each entry reports mean $\pm$ standard deviation over three independent trials.

Method	BC-VL #300	LiveVQA #245	MMSearch #171	MMS-Plus #311	SimpleVQA #300	VDR-Bench #200	WorldVQA #200	Avg.
Gemini-3-Flash	44.3 $\pm$ 0.8	73.5 $\pm$ 0.7	59.5 $\pm$ 0.9	22.5 $\pm$ 0.8	75.0 $\pm$ 0.6	14.5 $\pm$ 0.7	40.5 $\pm$ 0.8	47.5
ReAct	<u>54.7</u> $\pm$ 1.6	79.2 $\pm$ 1.4	74.1 $\pm$ 1.4	<u>37.3</u> $\pm$ 1.6	78.0 $\pm$ 1.1	<u>23.0</u> $\pm$ 1.4	57.0 $\pm$ 1.6	57.6
AgentFold	50.3 $\pm$ 1.4	82.0 $\pm$ 1.1	<u>77.6</u> $\pm$ 1.2	30.7 $\pm$ 1.5	73.7 $\pm$ 1.1	22.0 $\pm$ 1.1	56.0 $\pm$ 1.4	55.4
ContextFold	52.3 $\pm$ 1.4	<u>82.9</u> $\pm$ 1.1	76.6 $\pm$ 1.1	34.6 $\pm$ 1.4	<u>80.3</u> $\pm$ 0.9	21.0 $\pm$ 1.1	<u>57.5</u> $\pm$ 1.4	<u>57.7</u>
Ours	62.7 $\pm$ 1.1	85.4 $\pm$ 0.9	82.8 $\pm$ 1.1	40.3 $\pm$ 1.4	82.7 $\pm$ 0.9	28.5 $\pm$ 1.2	61.0 $\pm$ 1.1	63.2
GPT-5.2	47.0 $\pm$ 0.7	61.6 $\pm$ 0.9	46.8 $\pm$ 0.8	12.6 $\pm$ 0.7	67.3 $\pm$ 0.8	12.5 $\pm$ 0.6	35.5 $\pm$ 0.9	41.1
ReAct	55.7 $\pm$ 1.4	74.6 $\pm$ 1.6	69.8 $\pm$ 1.5	34.7 $\pm$ 1.4	78.0 $\pm$ 1.3	22.5 $\pm$ 1.6	55.5 $\pm$ 1.4	56.0
AgentFold	48.6 $\pm$ 1.2	75.6 $\pm$ 1.3	<u>75.4</u> $\pm$ 1.4	28.0 $\pm$ 1.3	72.4 $\pm$ 1.0	19.8 $\pm$ 1.3	53.7 $\pm$ 1.2	52.8
ContextFold	50.7 $\pm$ 1.3	76.3 $\pm$ 1.2	74.9 $\pm$ 1.3	31.9 $\pm$ 1.2	79.2 $\pm$ 1.1	18.7 $\pm$ 1.3	56.2 $\pm$ 1.2	55.2
Ours	61.3 $\pm$ 1.0	78.1 $\pm$ 1.1	80.7 $\pm$ 0.9	40.7 $\pm$ 1.2	81.3 $\pm$ 1.1	26.5 $\pm$ 1.0	59.5 $\pm$ 1.3	61.1
Qwen3.5-9B	25.3 $\pm$ 0.9	50.7 $\pm$ 0.6	19.3 $\pm$ 0.7	2.7 $\pm$ 0.5	54.3 $\pm$ 0.8	4.0 $\pm$ 0.6	20.0 $\pm$ 0.7	26.2
ReAct	41.8 $\pm$ 1.5	66.5 $\pm$ 1.2	62.4 $\pm$ 1.4	19.2 $\pm$ 1.5	67.8 $\pm$ 1.2	14.0 $\pm$ 1.2	44.5 $\pm$ 1.6	44.9
AgentFold	42.3 $\pm$ 1.3	69.0 $\pm$ 1.2	64.3 $\pm$ 1.5	18.2 $\pm$ 1.4	65.7 $\pm$ 1.3	13.5 $\pm$ 1.0	39.0 $\pm$ 1.2	44.3
ContextFold	<u>44.7</u> $\pm$ 1.4	<u>71.2</u> $\pm$ 1.0	<u>65.1</u> $\pm$ 1.2	<u>21.1</u> $\pm$ 1.3	<u>69.7</u> $\pm$ 1.0	<u>17.0</u> $\pm$ 1.2	<u>46.0</u> $\pm$ 1.2	<u>47.5</u>
Ours	54.3 $\pm$ 1.2	76.7 $\pm$ 1.1	71.3 $\pm$ 1.0	26.7 $\pm$ 1.2	74.7 $\pm$ 1.1	21.5 $\pm$ 1.1	49.5 $\pm$ 0.9	53.4
Qwen3.5-27B	28.7 $\pm$ 0.7	60.0 $\pm$ 0.9	25.7 $\pm$ 0.7	4.7 $\pm$ 0.7	61.3 $\pm$ 0.6	5.0 $\pm$ 0.6	21.5 $\pm$ 0.8	30.6
ReAct	48.0 $\pm$ 1.4	78.6 $\pm$ 1.5	74.3 $\pm$ 1.6	<u>33.0</u> $\pm$ 1.4	<u>76.7</u> $\pm$ 1.3	<u>23.0</u> $\pm$ 1.2	<u>52.0</u> $\pm$ 1.4	<u>54.8</u>
AgentFold	46.7 $\pm$ 1.5	77.6 $\pm$ 1.2	68.8 $\pm$ 1.2	21.3 $\pm$ 1.4	70.7 $\pm$ 1.2	18.5 $\pm$ 1.3	49.5 $\pm$ 1.2	49.9
ContextFold	51.7 $\pm$ 1.2	<u>80.3</u> $\pm$ 1.3	<u>75.8</u> $\pm$ 1.2	28.7 $\pm$ 1.2	72.7 $\pm$ 1.1	20.5 $\pm$ 1.0	50.5 $\pm$ 1.2	53.9
Ours	60.3 $\pm$ 1.0	83.5 $\pm$ 1.0	76.6 $\pm$ 0.9	38.3 $\pm$ 1.2	79.3 $\pm$ 1.0	27.0 $\pm$ 1.1	57.0 $\pm$ 1.3	60.3
Qwen3.5-35B-A3B	29.3 $\pm$ 0.6	62.0 $\pm$ 0.7	26.9 $\pm$ 0.9	4.3 $\pm$ 0.7	62.3 $\pm$ 0.6	8.0 $\pm$ 0.4	22.5 $\pm$ 0.7	31.7
ReAct	47.3 $\pm$ 1.5	72.5 $\pm$ 1.6	71.2 $\pm$ 1.3	28.3 $\pm$ 1.4	70.7 $\pm$ 1.4	20.5 $\pm$ 1.2	50.5 $\pm$ 1.4	<u>51.1</u>
AgentFold	43.3 $\pm$ 1.2	70.1 $\pm$ 1.4	67.4 $\pm$ 1.2	21.3 $\pm$ 1.1	68.3 $\pm$ 1.2	17.0 $\pm$ 1.0	46.0 $\pm$ 1.3	47.2
ContextFold	<u>48.6</u> $\pm$ 1.2	<u>75.8</u> $\pm$ 1.3	<u>73.5</u> $\pm$ 1.3	24.7 $\pm$ 1.3	<u>71.7</u> $\pm$ 1.1	18.5 $\pm$ 1.2	47.0 $\pm$ 1.1	51.0
Ours	57.3 $\pm$ 0.9	80.5 $\pm$ 1.1	76.2 $\pm$ 1.2	36.3 $\pm$ 1.1	77.7 $\pm$ 1.0	22.5 $\pm$ 1.2	56.5 $\pm$ 0.9	58.1

complexity. *Atomic factuality*: SimpleVQA [2] and WorldVQA [43] require the agent to recognize entities in images, ranging from common to long-tail cases, and retrieve the corresponding encyclopedic knowledge. *Dynamic information-seeking*: MMSearch [10] and LiveVQA [7] involve images related to recent events and specialized domains, requiring the agent to query the web for up-to-date or rare information. *Visual deep research*: BrowseComp-VL (BC-VL) [8], MMSearch-Plus (MMS-Plus) [23], and VDR-Bench [38] present questions where images provide key clues for identifying ambiguous or obfuscated entities. Solving them requires extracting fine-grained visual cues, conducting multi-step retrieval, and synthesizing cross-modal evidence to infer hard-to-find answers. Notably, MMSearch-Plus further involves multi-image queries where relevant clues are distributed across images. The per-benchmark instance counts are reported in Table 1: for BC-VL, WorldVQA, and VDR-Bench we evaluate on stratified subsets of the larger public pools that preserve the original difficulty and category structure, while the remaining four benchmarks use their released splits directly. All evaluation files are released in our repository.

Baselines. We compare MM-ContextFold with three types of baselines: *direct inference*, *ReAct*, and *context management methods*. Direct inference answers the multimodal query solely from the model’s parametric knowledge, without any external tool use. For *ReAct*, we adopt the standard ReAct framework as implemented in **WebWatcher** [8], a recent high-performing openly released multimodal deep research agent. For *context management methods*, we compare against two representative baselines, **AgentFold** [36] and **ContextFold** [22]. Both are designed for text-only scenarios, as context management for multimodal search agents remains largely

unexplored. For fair comparison, all methods use the same backbone, tool APIs, answer format, and total call budget. For AgentFold, we preserve its original multi-scale textual folding logic and expose the same multimodal query and *Visual Search* interface used by our method; the folded memories remain text-only. For ContextFold, we preserve its original branch-and-return controller and context inheritance rule; the method can call the same *Visual Search* tool, but because it has no explicit image-scoping mechanism, raw query images remain globally visible once placed in the main context and are therefore inherited by spawned branches.

Implementation. We evaluate all methods on five backbones via OpenRouter, namely Gemini-3-Flash, GPT-5.2, and the Qwen3.5 series (9B, 27B, 35B-A3B). All methods share the same tool set $\mathcal{U}$ (§3.1): *Text Retrieval* takes a free-form query generated by the agent and returns the top 10 results with titles, snippets, and URLs via the Serper API; *Visual Search* takes an image identifier α_k , resolves it to the image I_k via the registry $\mathcal{P}$, and submits I_k to the Google Lens API, which returns the top 10 visually similar results with their captions and webpages; *Web Visit* fetches the parsed content of a model-specified URL via Jina Reader. We cap interaction steps per branch at $B_{\max} = 5$ and total LLM calls at $T_{\max} = 20$; every main- and branch-state generation counts as one LLM call, so all methods operate under the same budget. We report accuracy averaged over three independent trials; run-to-run variation reflects both decoding nondeterminism and the changing responses of the live retrieval APIs. Correctness is determined by a fixed LLM-as-Judge protocol following prior work [8, 25]: the same judge model (Gemini-3-Flash) and prompt are used for all methods and backbones, reducing variance from answer-style differences.

Table 2: Ablation results (accuracy, %). Parenthesized values indicate the drop relative to MM-ContextFold on the same backbone.

Config	BC-VL	LiveVQA	MMSearch	MMS-Plus	SimpleVQA	VDR-Bench	WorldVQA	Avg.
MM-ContextFold (Gemini)	62.7	85.4	82.8	40.3	82.7	28.5	61.0	63.2
↪ Image Caption	49.0 (−13.7)	73.5 (−11.9)	65.5 (−17.3)	23.0 (−17.3)	64.7 (−18.0)	13.0 (−15.5)	39.5 (−21.5)	46.9 (−16.3)
w/o Init	53.3 (−9.4)	75.6 (−9.8)	76.6 (−6.2)	35.3 (−5.0)	73.3 (−9.4)	22.5 (−6.0)	53.5 (−7.5)	55.5 (−7.7)
w/o Image in Branch	52.7 (−10.0)	80.4 (−5.0)	76.0 (−6.8)	32.3 (−8.0)	76.7 (−6.0)	24.5 (−4.0)	55.5 (−5.5)	56.5 (−6.7)
w/ Image in Main	51.7 (−11.0)	78.6 (−6.8)	78.9 (−3.9)	35.0 (−5.3)	78.7 (−4.0)	20.5 (−8.0)	56.5 (−4.5)	56.8 (−6.4)
MM-ContextFold (Qwen3.5-35)	57.3	80.5	76.2	36.3	77.7	22.5	56.5	58.1
↪ Image Caption	40.0 (−17.3)	71.4 (−9.1)	40.4 (−35.8)	15.0 (−21.3)	53.0 (−24.7)	9.0 (−13.5)	23.5 (−33.0)	36.7 (−21.4)
w/o Init	39.0 (−18.3)	70.5 (−10.0)	63.2 (−13.0)	24.3 (−12.0)	60.7 (−17.0)	15.5 (−7.0)	46.5 (−10.0)	45.1 (−13.0)
w/o Image in Branch	44.3 (−13.0)	72.1 (−8.4)	65.1 (−11.1)	22.6 (−13.7)	67.7 (−10.0)	19.0 (−3.5)	51.5 (−5.0)	48.4 (−9.7)
w/ Image in Main	47.6 (−9.7)	76.7 (−3.8)	71.3 (−4.9)	22.7 (−13.6)	72.3 (−5.4)	17.5 (−5.0)	49.0 (−7.5)	50.6 (−7.5)

Table 3: Average accuracy (%) of fixed-step image-removal heuristics. $R@k$ denotes ReAct-Removal@ k , which removes the query images from step k onward.

Backbone	R@1	R@2	R@3	R@4	R@5	ReAct	Ours
Gemini	47.3	55.2	56.4	58.7	59.4	57.6	63.2
Qwen3.5-35	27.6	46.4	48.8	51.7	52.4	51.1	58.1

4.2 Main Results

Overall performance. As shown in Table 1, MM-ContextFold consistently outperforms all baselines on every backbone. On average across the five backbones, it reaches 59.2%, compared with 52.9% for ReAct, 53.1% for ContextFold, and 49.9% for AgentFold, corresponding to gains of +6.3, +6.1, and +9.3 percentage points (pp), respectively. The improvements are most pronounced on the three *Visual deep research* benchmarks, where direct inference yields only 18.7% accuracy on average and the tasks demand longer and more complex interaction trajectories (10.1 tool calls per trajectory on average vs. 6.5 on the other four benchmarks). On these three benchmarks, MM-ContextFold outperforms ContextFold, the strongest overall baseline, by 8.2 pp on average. In contrast, on the remaining four benchmarks, where direct inference already reaches 50.2% accuracy on average, MM-ContextFold’s gains over the three agentic retrieval baselines narrow (e.g., to +4.4 pp over ContextFold).

Comparison with context-management baselines. AgentFold progressively compresses earlier observations into multi-scale textual summaries. While this design improves context efficiency, visual-to-text compression may discard fine-grained details, and successive folding can further reduce the fidelity of summaries derived from earlier visual observations. Consistent with this, AgentFold trails ReAct baseline (49.9% vs. 52.9% on average). ContextFold provides branch-and-return control, but its inherited state remains modality-symmetric: once raw images enter the main context, they propagate into every subsequent branch regardless of whether the current subtask still requires them. MM-ContextFold instead adopts a modality-asymmetric inheritance pattern by keeping the persistent main state text-only and injecting only the selected raw images into a branch on demand. This targeted visual access yields gains over ContextFold of +5.5 to +7.1 pp across the five backbones.

4.3 Ablation Study

We conduct ablations on Gemini-3-Flash (Gemini) and Qwen3.5-35B-A3B (Qwen3.5-35) with four variants as shown in Table 2: **Image Caption** uses the initialization to textualize query images,

after which the agent executes subtasks with only *Text Retrieval* and *Web Visit*; **w/o Init** skips the initialization cycle (§3.3.2), leaving the agent in the main state planning only with the image identifiers $\mathcal{A}$; **w/o Image in Branch** disables raw image allocation to branch contexts, which rely solely on textualized results from initialization; **w/ Image in Main** injects raw images into the main context, removing the visual scoping constraint.

Ablation of the initialization cycle. Removing the initialization cycle (w/o Init) reduces accuracy by 7.7 pp on Gemini and 13.0 pp on Qwen3.5-35, with pronounced drops on benchmarks such as BrowseComp-VL and LiveVQA. The image descriptions produced during initialization enable the agent in the main state to decompose the task into subtasks. Planning from image identifiers $\mathcal{A}$ alone is not sufficient for these decisions.

Ablation of image exclusion in branch contexts. Both Image Caption and w/o Image in Branch exclude raw images from branch contexts. Image Caption incurs the largest drop among all variants (−16.3 pp on Gemini; −21.4 pp on Qwen3.5-35), as replacing raw images with captions deprives the agent of fine-grained details essential for accurate grounding. In contrast, w/o Image in Branch shows a smaller decline (−6.7 pp on Gemini; −9.7 pp on Qwen3.5-35), as the agent can still invoke *Visual Search* by specifying image identifiers $\alpha_k \in \mathcal{A}$ and can coarsely cross-validate returned metadata against the visual priors from initialization. Nevertheless, without direct pixel access, both variants underperform MM-ContextFold on tasks requiring fine-grained visual matching.

Ablation of visual scoping. The w/ Image in Main variant injects raw images into the main context, removing MM-ContextFold’s visual scoping constraint. Average accuracy drops by 6.4 pp on Gemini and 7.5 pp on Qwen3.5-35, falling back to nearly the level of ContextFold (Gemini: 56.8% vs. 57.7%; Qwen3.5-35: 50.6% vs. 51.0%). The degradation is moderate on *Atomic factuality* and *Dynamic information-seeking* benchmarks (e.g., −3.9 pp on MMSearch with Gemini) but more pronounced on *Visual deep research* tasks such as BrowseComp-VL (−11.0 pp on Gemini; −9.7 pp on Qwen3.5-35), where longer trajectories keep redundant visual tokens in the context for more steps. This aligns with the pilot-study observation (§3.2.3) that the marginal contribution of raw images diminishes after the early grounding steps.

Comparison with fixed-step image removal. We evaluate ReAct-Removal@ k ($R@k$), which drops the query images from step k onward. As shown in Table 3, $R@1$, where the agent never observes the raw images, severely degrades accuracy relative to ReAct (−10.3 pp on Gemini), underscoring the importance of images

Table 4: Token usage and tool calls averaged per sample across five backbones and seven benchmarks. Working Context denotes the maximum tokens passed to a single LLM call. Tool calls are broken down by type.

Metric	ReAct	AgentFold	ContextFold	Ours
Cumulative Tokens	54,551	15,993	49,594	44,088
Working Context	8,846	3,415	7,691	6,411
Tool Calls				
↪ <i>Text Retrieval</i>	7.4	4.4	5.4	5.8
<i>Web Visit</i>	1.5	1.8	2.2	1.4
<i>Visual Search</i>	0.7	0.6	1.1	0.9
Avg. Branches	-	-	2.7	2.3

Table 5: MMSearch-Plus: accuracy(%) and tool calls (*Visual Search* / total) by the number of query images, averaged over five backbones. Parenthesized counts denote the number of samples in each group.

Method	1 Image (222)		2 Images (54)		3+ Images (35)	
	Acc.	Tools	Acc.	Tools	Acc.	Tools
ReAct	31.8	0.9 / 10.1	28.8	0.6 / 9.8	24.6	0.7 / 9.2
AgentFold	24.9	0.7 / 7.3	21.4	1.2 / 8.2	20.9	2.2 / 9.2
ContextFold	30.2	0.7 / 5.3	24.1	2.1 / 12.4	22.8	2.7 / 16.3
MM-ContextFold	38.1	1.0 / 8.9	35.6	1.7 / 11.4	27.6	2.3 / 14.4

during early grounding. Accuracy then improves monotonically with k and surpasses ReAct at $k \geq 4$, indicating that image retention beyond the grounding phase on average hurts accuracy. Yet even the best heuristic (R@5) trails MM-ContextFold by 3.8 pp on Gemini and 5.7 pp on Qwen3.5-35. Since the optimal removal point varies across task types (Figure 2(c)), no single fixed k suits all tasks, motivating adaptive, per-subtask image allocation.

4.4 Token Usage Analysis

Table 4 and Figure 4 compare the four methods on token consumption and on accuracy as a function of trajectory length. Cumulative Tokens denotes the total API token cost per sample, and Working Context the maximum number of tokens passed to a single LLM call. AgentFold is the most economical on both metrics (16.0K / 3.4K), followed by MM-ContextFold (44.1K / 6.4K), which reduces the working context length by 27.5% relative to ReAct (54.6K / 8.8K), with ContextFold (49.6K / 7.7K) in between. Tool-call counts are comparable across methods, with *Visual Search* accounting for 0.6 to 1.1 calls. These token metrics, however, do not fully capture deployment cost: since MM-ContextFold introduces an initialization phase and branch-orchestration overhead, its lower working context reflects memory efficiency rather than a guaranteed wall-clock speedup. Figure 4(a) plots the working context against trajectory length. ReAct’s context grows almost linearly, whereas ContextFold and MM-ContextFold curb this growth through the state-switching mechanism, at the cost of an initially faster rise than ReAct caused by the longer structured prompts that folding requires. By confining raw images to branch contexts, MM-ContextFold further slows the growth at later steps. AgentFold maintains the smallest context throughout, but its accuracy drops markedly as trajectories lengthen (Figure 4(b)). Although accuracy declines on longer trajectories for all methods, MM-ContextFold consistently remains

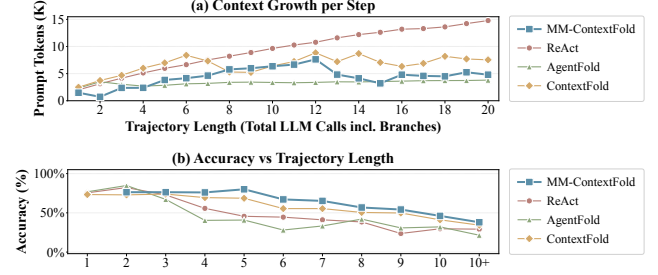


Figure 4: Context growth (a) and accuracy (b) as a function of trajectory length (total LLM calls), averaged across five backbones and seven benchmarks.

the most accurate, suggesting that the proposed framework better preserves task-relevant information over extended interactions.

4.5 Case Study

To examine how visual complexity affects each method, we group MMSearch-Plus samples by query-image count (Table 5) and report accuracy and tool calls (*Visual Search* / total). All methods degrade as image count increases, with MM-ContextFold retaining the highest accuracy in all three groups (38.1%, 35.6%, 27.6%). Meanwhile ReAct’s *Visual Search* usage stays nearly flat, whereas the other methods scale up to 2.2–2.7 calls, suggesting that it underutilizes visual tools on multi-image input. AgentFold yields the lowest accuracy in all three groups, indicating that its multi-scale compression discards fine-grained visual evidence that single- and multi-image queries alike depend on. ContextFold trails ReAct slightly on single-image queries (30.2% vs. 31.8%) but declines more sharply on multi-image ones (24.1%, 22.8%), with total tool calls rising from 5.3 to 16.3, consistent with the view that without visual scoping, globally visible images make later retrieval less targeted. In contrast, MM-ContextFold allocates images to branch contexts on demand, letting each branch focus on its subtask and fold findings back into the text-only main context, sustaining the highest accuracy at moderate tool-call cost.

5 Conclusion

This paper investigates the role of raw images in multimodal agentic retrieval tasks. A systematic empirical study of approximately 10,000 trajectories reveals that the agent’s reliance on raw images is concentrated in a limited number of visual-grounding steps, whose duration varies with task complexity, and that retaining images after their visual cues have been textualized brings little downstream benefit while increasing output entropy. Motivated by these findings, we propose MM-ContextFold, a training-free framework that maintains a persistent, text-only main context for high-level planning and confines raw images to ephemeral branch contexts that load them on demand. Experiments on seven MAR benchmarks across five backbone models show that MM-ContextFold improves average accuracy by 6.3 percentage points over the ReAct baseline while reducing working context length by 27.5%, with the largest gains on tasks that demand multi-step cross-modal evidence synthesis. In future work, we plan to explore adaptive mechanisms that detect when visual grounding is complete, and to extend the folding principle to other token-intensive modalities such as video.

References

- [1] Jiawei Chen, Xintian Shen, Lihao Zheng, Zhenwei Shao, Handong Cui, Chaoqun Du, Li Gong, Feng Gu, Xuefeng Hao, Wei He, et al. 2025. MindWatcher: Toward Smarter Multimodal Tool-Integrated Reasoning. *arXiv preprint arXiv:2512.23412* (2025).
- [2] Xianfu Cheng, Wei Zhang, Shiwei Zhang, Jian Yang, Xiangyuan Guan, Xianjie Wu, Xiang Li, Ge Zhang, Jiaheng Liu, Yuying Mai, et al. 2025. SimpleVQA: Multimodal Factuality Evaluation for Multimodal Large Language Models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 4637–4646.
- [3] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. In *Proceedings of the 28th European Conference on Artificial Intelligence (ECAI 2025)*. 2993–3000.
- [4] Yong Xien Chng, Tao Hu, Wenwen Tong, Xueheng Li, Jiandong Chen, Haojia Yu, Jiefan Lu, Hewei Guo, Hanming Deng, Chengjun Xie, et al. 2025. SenseNova-MARS: Empowering Multimodal Agentic Reasoning and Search via Reinforcement Learning. *arXiv preprint arXiv:2512.24330* (2025).
- [5] Zheng Chu, Xiao Wang, Jack Hong, Huiming Fan, Yuqi Huang, Yue Yang, Guohai Xu, Chenxiao Zhao, Cheng Xiang, Shengchao Hu, et al. 2026. REDSearcher: A Scalable and Cost-Efficient Framework for Long-Horizon Search Agents. *arXiv preprint arXiv:2602.14234* (2026).
- [6] Chenlong Deng, Mengjie Deng, Junjie Wu, Dun Zeng, Teng Wang, Qingsong Xie, Jiadeng Huang, Shengjie Ma, Changwang Zhang, Zhaoxiang Wang, et al. 2026. DeepImageSearch: Benchmarking Multimodal Agents for Context-Aware Image Retrieval in Visual Histories. *arXiv preprint arXiv:2602.10809* (2026).
- [7] Mingyang Fu, Yuyang Peng, Dongping Chen, Zetong Zhou, Benlin Liu, Yao Wan, Zhou Zhao, Philip S Yu, and Ranjay Krishna. 2025. Seeking and Updating with Live Visual Knowledge. *Advances in Neural Information Processing Systems* 38 (2025).
- [8] Xinyu Geng, Peng Xia, Zhen Zhang, Xinyu Wang, Qiuchen Wang, Ruixue Ding, Chenxi Wang, Jialong Wu, Kuan Li, Yida Zhao, et al. 2026. WebWatcher: Breaking New Frontiers of Vision-Language Deep Research Agent. In *International Conference on Learning Representations*, Vol. 2026. 61240–61271.
- [9] Soumya Suvra Ghosal, Youngseun Kim, Zhuowei Li, Ritwick Chaudhry, Linghan Xu, Hongjing Zhang, Jakub Zablocki, Yifan Xing, and Qin Zhang. 2026. VisRef: Visual Refocusing while Thinking Improves Test-Time Scaling in Multi-Modal Large Reasoning Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 33404–33414.
- [10] Dongzhi Jiang, Renrui Zhang, Ziyu Guo, Yanmin Wu, Pengshuo Qiu, Pan Lu, Zehui Chen, Guanglu Song, Gao Peng, Yu Liu, et al. 2025. MMSearch: Unveiling the Potential of Large Models as Multi-modal Search Engines. In *International Conference on Learning Representations*, Vol. 2025. 1662–1699.
- [11] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, et al. 2022. Language Models (Mostly) Know What They Know. *arXiv preprint arXiv:2207.05221* (2022).
- [12] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. 2023. Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation in Natural Language Generation. In *International Conference on Learning Representations*.
- [13] Kang-il Lee, Minbeom Kim, Seunghyun Yoon, Minsung Kim, Dongryeol Lee, Hyukhun Koh, and Kyomin Jung. 2025. V Lind-Bench: Measuring Language Priors in Large Vision-Language Models. In *Findings of the Association for Computational Linguistics: NAACL 2025*. 4129–4144.
- [14] Hongliang Li, Jiaxin Zhang, Wenhui Liao, Dezhi Peng, Kai Ding, and Lianwen Jin. 2025. RedundancyLens: Revealing and Exploiting Visual Token Processing Redundancy for Efficient Decoder-Only MLLMs. In *Findings of the Association for Computational Linguistics: ACL 2025*. 24056–24067.
- [15] Kuan Li, Zhongwang Zhang, Huifeng Yin, Liwen Zhang, Litu Ou, Jialong Wu, Wenbiao Yin, Baixuan Li, Zhengwei Tao, Xinyu Wang, et al. 2025. WebSailor: Navigating Super-human Reasoning for Web Agent. *arXiv preprint arXiv:2507.02592* (2025).
- [16] Xiaoxi Li, Jiajie Jin, Guanting Dong, Hongjin Qian, Yongkang Wu, Ji-Rong Wen, Yutao Zhu, and Zhicheng Dou. 2025. WebThinker: Empowering Large Reasoning Models with Deep Research Capability. *Advances in Neural Information Processing Systems* 38 (2025), 120091–120131.
- [17] Zhiyu Li, Shichao Song, Hanyu Wang, Simin Niu, Ding Chen, Jiawei Yang, Chenyang Xi, Huayi Lai, Jihao Zhao, Yezhaohui Wang, et al. 2025. MemOS: An Operating System for Memory-Augmented Generation (MAG) in Large Language Models. *arXiv preprint arXiv:2505.22101* (2025).
- [18] Zeping Li, Hongru Wang, Yiwen Zhao, Guanhua Chen, Yixia Li, Keyang Chen, Yixin Cao, Guangnan Ye, Hongfeng Chai, and Zhenfei Yin. 2026. Rethinking the Role of Entropy in Optimizing Tool-Use Behaviors for Large Language Model Agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 27955–27967.
- [19] Lingrui Mei, Jiayu Yao, Yuyao Ge, Yiwei Wang, Baolong Bi, Yujun Cai, Jiazhi Liu, Mingyu Li, Zhong-Zhi Li, Duzhen Zhang, et al. 2025. A Survey of Context Engineering for Large Language Models. *arXiv preprint arXiv:2507.13334* (2025).
- [20] Zile Qiao, Guoxin Chen, Xuanzhong Chen, Donglei Yu, Wenbiao Yin, Xinyu Wang, Zhen Zhang, Baixuan Li, Huifeng Yin, Kuan Li, et al. 2025. WebResearcher: Unleashing Unbounded Reasoning Capability in Long-Horizon Agents. *arXiv preprint arXiv:2509.13309* (2025).
- [21] Hai-Long Sun, Zhun Sun, Houwen Peng, and Han-Jia Ye. 2025. Mitigating Visual Forgetting via Take-along Visual Conditioning for Multi-modal Long CoT Reasoning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 5158–5171.
- [22] Weiwei Sun, Miao Lu, Zhan Ling, Kang Liu, Xuesong Yao, Yiming Yang, and Jiecao Chen. 2026. Scaling Long-Horizon Agent via Context Folding. In *Proceedings of the 43rd International Conference on Machine Learning*, Vol. 306.
- [23] Xijia Tao, Teng Yihua, Xinxing Su, Xinyu Fu, Jihao Wu, Chaofan Tao, Ziru Liu, Haoli Bai, Rui Liu, and Lingpeng Kong. 2026. MMSearch-Plus: Benchmarking Provenance-Aware Search for Multimodal Browsing Agents. In *International Conference on Learning Representations*, Vol. 2026. 70385–70422.
- [24] MiroMind Team, Song Bai, Lidong Bing, Carson Chen, Guanzheng Chen, Yuntao Chen, Zhe Chen, Ziyi Chen, Jifeng Dai, Xuan Dong, et al. 2025. MiroThinker: Pushing the Performance Boundaries of Open-Source Research Agents via Model, Context, and Interactive Scaling. *arXiv preprint arXiv:2511.11793* (2025).
- [25] Tongyi DeepResearch Team, Baixuan Li, Bo Zhang, Dingchu Zhang, Fei Huang, Guangyu Li, Guoxin Chen, Huifeng Yin, Jialong Wu, Jingren Zhou, et al. 2025. Tongyi DeepResearch Technical Report. *arXiv preprint arXiv:2510.24701* (2025).
- [26] Xinyu Tian, Shu Zou, Zhaoyuan Yang, Mengqi He, Fabian Waschkowski, Lukas Wesemann, Peter Tu, and Jing Zhang. 2026. More Thought, Less Accuracy? On the Dual Nature of Reasoning in Vision-Language Models. In *International Conference on Learning Representations*, Vol. 2026. 133928–133965.
- [27] Jiacong Wang, Zijian Kang, Haochen Wang, Xiao Liang, Ya Wang, Jiawen Li, Bohong Wu, Jiao Ran, Haiyong Jiang, Chao Feng, et al. 2026. VGR: Visual Grounded Reasoning. In *International Conference on Learning Representations*, Vol. 2026. 89506–89528.
- [28] Qiuchen Wang, Shihang Wang, Yu Zeng, Qiang Zhang, Fanrui Zhang, Zhuoning Guo, Bosi Zhang, Wenxuan Huang, Lin Chen, Zehui Chen, et al. 2026. Vim-RAG: Navigating Massive Visual Context in Retrieval-Augmented Generation via Multimodal Memory Graph. *arXiv preprint arXiv:2602.12735* (2026).
- [29] Jialong Wu, Baixuan Li, Runnan Fang, Wenbiao Yin, Liwen Zhang, Zhenglin Wang, Zhengwei Tao, Ding-Chu Zhang, Zekun Xi, Robert Tang, et al. 2025. WebDancer: Towards Autonomous Information Seeking Agency. *Advances in Neural Information Processing Systems* 38 (2025), 120957–120985.
- [30] Jialong Wu, Wenbiao Yin, Yong Jiang, Zhenglin Wang, Zekun Xi, Runnan Fang, Linhai Zhang, Yulan He, Deyu Zhou, Pengjun Xie, et al. 2025. WebWalker: Benchmarking LLMs in Web Traversal. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 10290–10305.
- [31] Xixi Wu, Kuan Li, Yida Zhao, Liwen Zhang, Litu Ou, Huifeng Yin, Zhongwang Zhang, Xinniao Yu, Dingchu Zhang, Yong Jiang, et al. 2025. ReSum: Unlocking Long-Horizon Search Intelligence via Context Summarization. *arXiv preprint arXiv:2509.13313* (2025).
- [32] Rui Xu, Yunke Wang, Yong Luo, and Bo Du. 2026. Rethinking Visual Token Reduction in LVLs Under Cross-Modal Misalignment. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 27323–27331.
- [33] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-Mem: Agentic Memory for LLM Agents. *Advances in Neural Information Processing Systems* 38 (2025), 17577–17604.
- [34] Zhaorui Yang, Bo Pan, Han Wang, Yiyao Wang, Xingyu Liu, Luoxuan Weng, Yingchaojie Feng, Haozhe Feng, Minfeng Zhu, Bo Zhang, et al. 2026. Multimodal DeepResearcher: Generating Text-Chart Interleaved Reports from Scratch with Agentic Framework. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 34368–34377.
- [35] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*.
- [36] Rui Ye, Zhongwang Zhang, Kuan Li, Huifeng Yin, Zhengwei Tao, Yida Zhao, Liangcai Su, Liwen Zhang, Zile Qiao, Xinyu Wang, et al. 2026. AgentFold: Long-Horizon Web Agents with Proactive Context Folding. In *International Conference on Learning Representations*, Vol. 2026. 154754–154776.
- [37] Hongli Yu, Tinghong Chen, Jiangtao Feng, Jiangjie Chen, Weinan Dai, Qiyang Yu, Ya-Qin Zhang, Wei-Ying Ma, Jingjing Liu, Mingxuan Wang, et al. 2026. MemAgent: Reshaping Long-Context LLM with Multi-Conv RL-based Memory Agent. In *International Conference on Learning Representations*, Vol. 2026. 39458–39486.
- [38] Yu Zeng, Wenxuan Huang, Zhen Fang, Shuang Chen, Yufan Shen, Yishuo Cai, Xiaomian Wang, Zhenfei Yin, Lin Chen, Zehui Chen, et al. 2026. Vision-DeepResearch Benchmark: Rethinking Visual and Textual Search for Multimodal Large Language Models. *arXiv preprint arXiv:2602.02185* (2026).
- [39] Ruiyang Zhang, Qianguo Sun, Chao Song, Yiyang Qi, and Zhedong Zheng. 2026. VSearcher: Long-Horizon Multimodal Search Agent via Reinforcement Learning. *arXiv preprint arXiv:2603.02795* (2026).
- [40] Yongyue Zhang and Yaxiong Wu. 2026. MLDocRAG: Multimodal Long-Context Document Retrieval Augmented Generation. *arXiv preprint arXiv:2602.10271* (2026).

- [41] Yuxiang Zheng, Dayuan Fu, Xiangkun Hu, Xiaojie Cai, Lyumanshan Ye, Pengrui Lu, and Pengfei Liu. 2025. DeepResearcher: Scaling Deep Research via Reinforcement Learning in Real-world Environments. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 414–431.
- [42] Jingqi Zhou, Sheng Wang, Jingwei Dong, Kai Liu, Lei Li, Jiahui Gao, Jiyue Jiang, Lingpeng Kong, and Chuan Wu. 2025. ProReason: Multi-Modal Proactive Reasoning with Decoupled Eyesight and Wisdom. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 31662–31691.
- [43] Runjie Zhou, Youbo Shao, Haoyu Lu, Bowei Xing, Tongtong Bai, Yujie Chen, Jie Zhao, Lin Sui, Haotian Yao, Zijia Zhao, et al. 2026. WorldVQA: Measuring Atomic World Knowledge in Multimodal Large Language Models. *arXiv preprint arXiv:2602.02537* (2026).
- [44] Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Bryan Kian Hsiang Low, and Paul Liang. 2026. MEM1: Learning to Synergize Memory and Reasoning for Efficient Long-Horizon Agents. In *International Conference on Learning Representations*, Vol. 2026. 58413–58438.

A Dataset

This section provides details on the seven benchmarks used in this paper, complementing the summary in §4.1. We first present the benchmark taxonomy and category assignment rationale (Table 6), then describe the unified task interface and report benchmark sizes (Table 7), and finally document the data source and sampling policy for each dataset (Table 8).

A.1 Category Assignment Rationale

As summarized in Table 6, our three-way grouping is motivated by the role that visual grounding plays in the subsequent retrieval trajectory. *Atomic factuality* tasks usually require a relatively short chain: once the image helps identify the referent, the remaining work is largely factual lookup. *Dynamic information-seeking* tasks additionally require fresh or niche web evidence, but the image still mainly acts as an anchor for retrieving the correct entity or event. In contrast, *Visual deep research* tasks require multiple rounds of disambiguation, verification, and evidence composition, and therefore place a heavier burden on both visual grounding and long-horizon context management.

A.2 Unified Task Interface

All seven benchmarks are normalized into a common MAR interface consisting of a natural-language question q , an ordered list of query images $\mathcal{I}$, and a gold answer a^* . If a benchmark provides answer options, the options are appended to the question text; otherwise the agent produces a free-form short answer. This normalization enables all methods to share the same tool interface, answer slot, stopping rule, and evaluation protocol.

As shown in Table 7, the reported benchmark collection contains 1,727 samples in total under this reporting protocol. Among the seven benchmarks, MMSearch-Plus is the only one with frequent multi-image queries. In the full normalized MMSearch-Plus release of 311 examples, 222 are single-image questions and 89 are multi-image questions (54 with two images, 30 with three, 4 with four, and 1 with five), yielding 1.42 query images per sample on average.

A.3 Data Source and Sampling Policy

Table 8 summarizes how each reported set is obtained. The key distinction is between (i) datasets for which we directly adopt the released benchmark split used in prior multimodal-agent evaluations, and (ii) datasets for which we construct a smaller evaluation subset from a larger released pool using dataset-specific sampling policies based on the available difficulty/category metadata.

Comparison with WebWatcher benchmark usage. Among the benchmarks in our paper, MMSearch, LiveVQA, SimpleVQA, and BC-VL overlap with the benchmark interface released by WebWatcher [8]. For MMSearch, LiveVQA, and SimpleVQA, we follow the released evaluation split directly. For BrowseComp-VL, we construct a 300-example evaluation set from the released 400-example WebWatcher pool by retaining all 200 Level-2 examples and sampling 100 examples from the 200 Level-1 pool. By contrast, WorldVQA and VDR-Bench are not part of the original WebWatcher benchmark suite, and our 200-example evaluation sets are new proportional subsets constructed from the larger public WorldVQA and VDR-Bench pools.

Sampling strategy for constructed subsets. For datasets where we construct a smaller evaluation subset, we do not sample uniformly at random. Instead, we preserve the benchmark structure exposed by the released metadata. For BrowseComp-VL, we retain all 200 Level-2 examples and sample 100 of the 200 Level-1 examples, so that both difficulty levels remain represented and the harder level is kept in full while the evaluation size stays manageable. For WorldVQA, this means proportional sampling over the eight non-People categories used in our evaluation. For VDR-Bench, this means preserving both difficulty buckets (e.g., base, hard, hard1) and topical categories encoded in the public example IDs. This design reduces the risk that the sampled subset over-represents only easy categories, only hard categories, or otherwise distorts the released benchmark structure.

Data release. All normalized benchmark files used in this paper, including the directly adopted splits, the sampled evaluation subsets, and the corresponding sample IDs, are released in the public repository referenced in the main paper.

B Implementation Details

This section supplements the implementation summary in §4.1 with reproducibility-relevant details omitted from the main text for space.

B.1 Infrastructure and Hyperparameters

Table 9 lists the five backbone models used in the main benchmark evaluations, which are accessed through the OpenRouter API. The local deployment used for the entropy probe and paired image-removal analyses is described below. We use temperature = 0.2 and top_p = 0.95 for all backbones; no system-level sampling changes are made across methods or benchmarks.

Local deployment for entropy analysis. The entropy probe (§3.2.2) requires token-level log-probabilities unavailable from proprietary APIs. We therefore deploy the three Qwen3.5 backbones locally on 8×NVIDIA H20 GPUs (96 GB each) using SGLang with 4-way tensor parallelism for every model. All local models are served with the same sampling parameters as the OpenRouter runs (temperature = 0.2, top-p = 0.95) and return top-20 log-probabilities per token for entropy computation. Each of the three Qwen3.5 backbones is run on the full 1,727-sample benchmark suite; after excluding samples where the agent produced no valid trajectory (e.g., immediate parsing failure), 5,087 trajectories remain and are used in the entropy analysis, as detailed in Table 10.

B.2 Tool-Call Formats and Baseline Adaptation

Although all methods share the same three retrieval tools and the same total call budget, their action grammars differ because the baselines use different prompt and parsing protocols. To ensure a fair comparison, we preserve each method’s native tool-call format and parsing logic while unifying the underlying tool implementations. Table 11 summarizes the key differences.

Table 6: Benchmark taxonomy used in this work. We summarize the aspects of each benchmark that are most relevant to multimodal agentic retrieval: how the query image contributes, how much external retrieval is required, and why the benchmark belongs to a particular category.

Benchmark	Category	Image Regime	Role of the Query Images	External Retrieval
SimpleVQA	Atomic factuality	Single image	Identifies an entity, artifact, landmark, or product whose name must be recognized.	Answer depends on encyclopedic or attribute knowledge not reliably stored in parametric memory.
WorldVQA	Atomic factuality	Single image	Provides the entity to be identified, often involving long-tail objects, places, or culturally specific items.	Retrieval is needed to ground rare or tail knowledge once the visual referent is identified.
MMSearch	Dynamic info-seeking	Single image	Anchors a current event, organization, scene, or object that must first be identified.	Answer depends on fresh or specialized web information, making static memorization insufficient.
LiveVQA	Dynamic info-seeking	Single image	Serves as an entry point to a time-sensitive or domain-specific query.	Required evidence often comes from recent webpages or niche sources that must be actively verified.
BC-VL	Visual deep research	Single image	Contains a key clue, but the referent is visually ambiguous, indirect, or difficult to name from a single glance.	Requires chained retrieval and cross-source synthesis rather than one-shot identification.
MMS-Plus	Visual deep research	Single/multi-image	In multi-image queries, relevant clues are distributed across images; the agent must determine which image matters for which subtask.	Needs to combine evidence across images and sources, not just identify each clue.
VDR-Bench	Visual deep research	Fine-grained cue	Contributes subtle evidence such as text fragments, layout cues, or object-level details supporting a longer reasoning chain.	Final answer emerges only after multi-step retrieval, verification, and synthesis over heterogeneous evidence.

Table 7: Reported benchmark sizes and image statistics. The reported set is either the released benchmark split or a stratified subset from the public pool. For MMS-Plus, statistics are over the full normalized release.

Benchmark	#Samples	Avg. #Imgs	Max #Imgs
SimpleVQA	300	1.00	1
WorldVQA	200	1.00	1
MMSearch	171	1.00	1
LiveVQA	245	1.00	1
BC-VL	300	1.00	1
MMS-Plus	311	1.42	5
VDR-Bench	200	1.00	1
Total	1,727	–	–

ReAct / AgentFold (JSON):

```
<tool_call>
{"name": "lens_search",
 "arguments": {"image_id": "IMG_001"}} </tool_call>
```

ContextFold (XML-style):

```
<function=lens_search>
<parameter=image_id>IMG_001</parameter> </function>
```

Preserving action grammars. Following the adaptation policy in §4.1, we preserve each baseline’s context-management logic and action grammar while exposing the shared multimodal inputs and tool interfaces. Parsed tool names and arguments are routed to the same backend APIs (Serper, Jina Reader, and Google Lens). This controls tool access while retaining the method-specific prompts and parsers. As a concrete example, invoking *Visual Search* on image IMG_001 is expressed differently across methods, yet both are routed to the same Google Lens API call.

Our format choice. For MM-ContextFold, we adopt the JSON-based `<tool_call>` format that has become the *de facto* standard in recent agentic retrieval systems [5, 8, 15, 35]. Compared with ContextFold’s XML-style tags, JSON tool calls are natively supported by most LLM serving frameworks and align with the function-calling conventions of mainstream model providers, reducing parsing overhead and backbone-specific adaptation.

B.3 Branch Execution and Folding

This subsection describes implementation-level details that go beyond the formal definitions in §3.3.1–3.3.3.

Tool-use pattern within a branch. When a regular retrieval branch receives non-empty assigned_images, it begins with `lens_search` for visual grounding, then typically shifts to `search` and `visit` once a candidate entity or clue is grounded. Each tool call carries a goal field specifying the retrieval intent, reducing redundant calls. This staged pattern aligns with the empirical

Table 8: Data source and sampling policy. “Pool” denotes the released source set available after normalization. “Reported” denotes the set size used in this paper.

Benchmark	Source	Pool	Reported	Sampling Policy
SimpleVQA	[2], WebWatcher [8]	300	300	Used directly without additional sampling.
MMSearch	[10], WebWatcher [8]	171	171	Used directly without additional sampling.
LiveVQA	[7], WebWatcher [8]	245	245	Used directly without additional sampling.
MMS-Plus	[23]	311	311	Used directly after normalization. Image statistics computed over the full release preserving category proportions.
BC-VL	WebWatcher [8]	400	300	All 200 Level-2 examples adopted directly; 100 Level-1 examples sampled from the 200 Level-1 pool.
WorldVQA	[43]	3,000	200	Stratified subset by us. Proportional sampling across the 8 non-People categories preserving original ratios.
VDR-Bench	[38]	2,000	200	Stratified subset by us. Preserves released difficulty buckets and topical categories proportionally.

Table 9: Backbone models, serving configuration, and shared hyperparameters.

Backbone models	
Gemini-3-Flash	OpenRouter API
GPT-5.2	OpenRouter API
Qwen3.5-9B	OpenRouter API
Qwen3.5-27B	OpenRouter API
Qwen3.5-35B-A3B	OpenRouter API
Shared hyperparameters	
Temperature / Top- p	0.2 / 0.95
$T_{\max}$ (total LLM calls)	20 (all methods)
$B_{\max}$ (max branch steps)	5 (MM-ContextFold/ContextFold)
Top- k retrieval results	10 (<i>Text Retrieval/Visual Search</i>)
Independent trials	3

Table 10: Trajectory counts for the local entropy analysis. Each backbone is run on all 1,727 samples; invalid trajectories are excluded.

Backbone	Total	Excluded	Valid
Qwen3.5-9B	1,727	45	1,682
Qwen3.5-27B	1,727	21	1,706
Qwen3.5-35B-A3B	1,727	28	1,699
Total	5,181	94	5,087

finding in Figures 1 and 2: raw images are most useful at the beginning of a subtask, while later steps are text-dominant.

Folding boundary. The RETURN payload is plain text summarizing the current hypothesis, supporting evidence, and remaining uncertainty. Of the branch-local contents, only this return message is written to the persistent main context; the full branch-local multimodal trajectory is discarded. The main-state reasoning and the corresponding BRANCH action are also retained, as specified in

Table 11: Tool-call format comparison. Each method retains its action grammar; all are routed to the same backend tool implementations.

Method	Tool-call format
ReAct(WebWatcher)	JSON inside <code><tool_call></code> tags. Parameter key: "arguments". { "name": "search", "arguments": { "queries": ["... "] }}
AgentFold	Same JSON <code><tool_call></code> syntax as ReAct. Parameter key: "arguments". Additionally uses <code><compress></code> and <code><motivation></code> blocks for multi-scale memory folding.
ContextFold	XML-style tags: <code><function=name></code> with nested <code><parameter=key>value</parameter></code> . Parsed into a flat string-valued argument dict.
MM-Context- Fold	Main state: <code><subtask></code> containing a JSON block with description, prompt, and assigned_images. Branch state: JSON inside <code><tool_call></code> tags. Parameter key: "parameters".

Eq. (6). If the branch budget $B_{\max}$ is exhausted before an explicit RETURN, the final call is constrained to summarize partial findings and terminate.

Error handling. Malformed model outputs (e.g., missing action fields, batched tool calls) trigger a correction re-prompt that feeds the malformed output back to the model with an error message. The re-prompt counts toward the branch budget $B_{\max}$ and the global budget $T_{\max}$. Figure 5 illustrates two consecutive branch cycles on the same query after initialization, contrasting a visual-grounding branch ($\mathcal{A}_t \neq \emptyset$) with a text-only verification branch ($\mathcal{A}_t = \emptyset$). Algorithm 1 summarizes the overall inference-time control flow.

Cycle 1 – visual grounding

Main → **BRANCH**:

```
{
  "description": "identify the TV program
  shown in the screenshot",
  "prompt": "Identify the source program and
  episode from <image_0>.",
  "assigned_images": ["<image_0>"]}

```

Branch (with <image_0> loaded):

- ① `lens_search(<image_0>) → PBS, Singapore ...`
- ② **RETURN**: “Candidate: PBS *Singapore*:
 Designing a Megacity in Harmony with Nature.”

Cycle 2 – text-only verification

Main → **BRANCH**:

```
{
  "description": "verify the air date of the
  candidate episode",
  "prompt": "Confirm the original air date of
  the PBS episode identified above.",
  "assigned_images": []}

```

Branch (no image loaded):

- ① `search("PBS Singapore .... air date")`
- ② `visit(https://www.pbs.org/...)`
- ③ **RETURN**: “Confirmed. First aired August 17, 2023”

Figure 5: Worked example showing two consecutive branch cycles: the first loads a query image for visual grounding; the second assigns no image and performs text-only verification.

Algorithm 1 Inference-time control flow of MM-ContextFold.

- 1: Assign each query image I_k a stable identifier α_k , build the registry $\mathcal{P} = \{(\alpha_k, I_k)\}$, and initialize the main context $\tau_1 = (q, \mathcal{A})$ with $\mathcal{A} = \{\alpha_k\}$.
- 2: Initialization cycle: issue **BRANCH**($s_{\text{init}}, \mathcal{A}$), which loads all query images and returns $e_{\text{init}} = \{d_k\}$, one textual description per image; fold e_{init} into the main context.
- 3: **while** answer not finalized and global budget T_{max} not exhausted **do**
- 4: Main state emits **FINISH**(a) or **BRANCH**($s_t, \mathcal{A}_t$).
- 5: **if** **FINISH**(a) **then**
- 6: Terminate and output a .
- 7: **else**
- 8: Create branch: inherit the text-only main context, load only the images in $\mathcal{A}_t$.
- 9: Roll out the branch for at most B_{max} steps; each step emits a tool call or **RETURN**(e_t).
- 10: Fold e_t into the main context; discard the branch trajectory.
- 11: **end if**
- 12: **end while**

B.4 Empirical Analysis Protocol

The methodology of the three pilot analyses (grounding annotation, entropy probe, and image-removal ablation) is described in §3.2.1–§3.2.3. Below we document implementation-level details not covered in the main text.

Grounding annotation pipeline. The annotator is Gemini-3-Flash, prompted with the original question, the query images, and the trajectory steps in chronological order; each step r_t is labeled with respect to the text of the preceding steps, following the

counterfactual rule stated in the prompt. The three mutually exclusive categories used in §3.2.1 are obtained from two binary labels produced in two passes: `image_needed` (whether the step extracts new visual information directly from the raw images) and `image_info_used` (whether the step uses any image-derived information at all). *Image Needed* corresponds to `image_needed=true`, *Image Derived* to `image_needed=false` with `image_info_used=true`, and *Text Only* to both labels being false. Three human annotators independently labeled a random subsample of 500 steps; the LLM annotations achieve 93.5% agreement with the majority human vote. The annotation prompts are provided in Appendix E.

Entropy computation details. The full-vocabulary entropy $H(\tau_t)$ defined in §3.2.2 is estimated from the top-20 log-probabilities returned per token: the residual probability mass $p_{\text{rest}} = 1 - \sum_{i=1}^{20} p_i$ is treated as a single tail bin contributing $-p_{\text{rest}} \log p_{\text{rest}}$, so the reported values are a grouped (lower-bound) estimate of the exact vocabulary-level entropy, computed identically for τ_t and τ_t^{IR} . For thinking-enabled Qwen3.5 backbones, we compute entropy only over tokens inside the `<think>...</think>` reasoning segment, excluding the final answer tokens; this ensures that ΔH_t reflects changes in the model’s internal reasoning distribution rather than answer formatting.

B.5 Scope of Visual Analysis

Our study focuses on the retention of *original query images*. Unlike retrieved images that enter the context at varying points depending on when the agent invokes Visual Search, query images are present from step 1 and shared uniformly across all samples, making them the cleanest unit for isolating a context-management question: once a visual clue has been extracted and verbalized, should the raw input image still remain globally visible?

In our ReAct-based analysis, retrieved images from *Visual Search* are excluded as the primary analysis unit for three reasons. First, their content is non-deterministic and search-engine dependent: the same query image may yield different retrieved results across runs due to ranking changes, freshness, and index updates, making controlled before-and-after comparisons unreliable. Second, analyzing retrieved-image retention would conflate two factors we wish to keep separate: *retrieval quality* (whether the search engine returns relevant images) and *context-retention policy* (whether keeping those images in subsequent steps helps or hurts reasoning). Third, retrieved images enter the ReAct context at varying trajectory positions depending on when the agent invokes Visual Search, whereas query images are present from step 1 onward and uniformly shared across all samples. This uniformity is essential for the step-level aggregation in Figures 1 and 2. By focusing on query-image retention, we obtain a cleaner intervention with a fixed, sample-independent baseline for studying when persistent raw visual context helps and when it becomes redundant.

B.6 Evaluation Protocol

The evaluation pipeline is shared across all methods and backbones; we describe here the implementation details beyond the summary in §4.1.

Judge model and prompt. We use Gemini-3-Flash (accessed via OpenRouter) as the judge model for all evaluations. The judge

prompt follows the WebWatcher and Qwen-DeepResearch official evaluation template [8, 25], which performs a two-step procedure: (1) *extract* the core factual claim from the model’s response, and (2) *grade* it against the gold answer as one of three classes: CORRECT (semantically equivalent, ignoring casing, phrasing, and minor typos), INCORRECT (contains a factual contradiction), or NOT_ATTEMPTED (no clear claim is made). The judge may emit either the textual grades above or the equivalent letter codes A/B/C, which are mapped to the same three classes by a deterministic parser. The letter codes are mapped as A=CORRECT, B=INCORRECT, and C=NOT_ATTEMPTED. The full judge prompt is provided in Appendix E.

Grading rules. For numerical answers, values correct to the last significant figure of the gold target are accepted. When the prediction only addresses the specific question asked, extra detail in the gold target is not penalized. If a response contains both correct and incorrect claims, the grade is INCORRECT.

Handling failed trajectories. If a trajectory terminates with status other than success (e.g., the agent exhausted its budget without producing a final answer) or returns an empty answer, the sample is automatically graded as NOT_ATTEMPTED without invoking the judge model.

Parallelism and reproducibility. Judge calls are parallelized with up to 8 concurrent workers. The judge uses temperature = 0 to maximize grading consistency. Table 1 reports mean accuracy and standard deviation over three independent runs; the supplementary HLE-VL table reports the means. Accuracy is the fraction of evaluated samples graded CORRECT; INCORRECT and NOT_ATTEMPTED samples remain in the denominator.

C Comparison With Context Management

This section provides a principled comparison of MM-ContextFold with the context-management baselines. We organize the analysis around the three design principles introduced in §3.3: (P1) *decoupled reasoning and grounding*, (P2) *ephemeral visual context*, and (P3) *adaptive image allocation*.

C.1 Adaptation Principle

We adapt all baselines to the same MAR setting under a strict policy: *preserve each method’s original context-management logic and action grammar while unifying the external interfaces* (backbone, tool APIs, answer format, and global call budget $T_{\max} = 20$). The shared interfaces control tool access, while method-specific prompts implement the different context-management designs.

ReAct (WebWatcher). We directly adopt the publicly released WebWatcher agent, which follows the standard ReAct loop. The agent receives the original question and all query images at step 1, and appends all tool observations (including retrieved images) to a single, ever-growing context. No context-management modification is applied; this serves as the prevailing MAR baseline.

AgentFold. We preserve AgentFold’s original multi-scale textual compression logic: after each tool call, the agent decides whether to compress earlier steps into a textual summary. To adapt it to MAR, we expose the same query-image input and Visual Search tool interface as our method while keeping the compressed memory text-only, faithful to the original design. This means visual information

can only survive through textualization, a principled design that is inherently lossy when fine-grained visual details are needed later.

ContextFold. We preserve ContextFold’s original branch-and-return controller, inherited-state design, and its native XML-style tool-call format (see §B.2). The agent can invoke the same Visual Search tool, but several structural differences distinguish it from MM-ContextFold:

- **Per-subtask image allocation.** In ContextFold, raw images present in the main state are automatically inherited by all spawned branches regardless of whether the current subtask requires visual grounding. MM-ContextFold instead lets the main state declare $\mathcal{A}_t \subseteq \mathcal{A}$ (possibly $\emptyset$) per branch, so that later verification branches run text-only without carrying irrelevant image tokens.
- **Modality-symmetric inheritance.** ContextFold’s inherited state is a full copy of the main context including any raw images, making it modality-symmetric. MM-ContextFold enforces modality-*asymmetric* inheritance: only textual artifacts are inherited from the main state, and raw images are injected on demand.

C.2 Principle-Level Analysis

P1: Decoupled reasoning and grounding. ReAct interleaves planning, grounding, and retrieval in a single context with no structural separation. AgentFold partially decouples them by compressing earlier observations, but the compression itself mixes visual and textual information. ContextFold provides subtask isolation via branches, but within each branch planning and grounding are still interleaved, and the main state can contain raw images. MM-ContextFold enforces a separation: the main state performs planning over text-only artifacts, and visual grounding is confined to branches.

P2: Ephemeral visual context. MM-ContextFold guarantees that no raw image persists in the main context after a branch returns. In ReAct, images accumulate indefinitely. In ContextFold, images inherited from the main state persist across branches. AgentFold achieves partial ephemerality by compressing images into text, which is a one-way, lossy transformation and is not a reversible scoping mechanism.

P3: Adaptive image allocation. In MM-ContextFold, each subtask explicitly declares which images it needs via the assigned_images field, including $\mathcal{A}_t = \emptyset$ for purely textual subtasks. This per-subtask allocation reflects the observation that different stages of a trajectory have different visual demands (Figure 2(a)): early grounding steps benefit from direct image access, while later verification steps typically do not. By contrast, ReAct keeps all images always visible, AgentFold replaces them with compressed text, and ContextFold inherits them globally, each handling image lifecycle at a coarser granularity than per-subtask allocation.

D More Results

D.1 Results on HLE-VL

To further validate MM-ContextFold on a benchmark outside our main evaluation suite, we report results on **Humanity’s Last Exam-Visual (HLE-VL)** [8], a highly challenging multimodal benchmark that spans biology, chemistry, CS/AI, engineering, humanities, mathematics, physics, and other disciplines. HLE-VL is

Table 12: Design-principle comparison. Each method is assessed against the three principles (P1–P3) motivating MM-ContextFold. ✓ = fully satisfied, ~ = partially satisfied, ✗ = not satisfied. All agentic methods share the backbone, tools, and budget described in §4.1.

Method	P1	P2	P3	How it manages visual context
ReAct	✗	✗	✗	Append-only mixed-modality history. Raw query images remain globally visible throughout the trajectory; planning and grounding share a single interleaved context.
AgentFold	~	~	✗	Multi-scale textual compression. Planning operates over compressed text (partial P1), and raw images are replaced by textual summaries (partial P2). Compression is lossy, so fine-grained visual cues may be lost and cannot be selectively reloaded later.
ContextFold	~	✗	✗	Branch-and-return with modality-symmetric inheritance. Branching provides subtask isolation (partial P1), but raw images in the main context propagate into every subsequent branch, and image assignment is implicit rather than per-subtask.
MM-ContextFold	✓	✓	✓	Text-only main state for planning (P1); raw images confined to ephemeral branches and discarded after return (P2); each BRANCH action explicitly specifies which images to load, including $\emptyset$ for text-only subtasks (P3).

Table 13: Results on HLE-VL (accuracy, %). Each entry reports the mean over three independent runs. All agentic methods use the same tool set and budget.

Backbone	Direct	ReAct	AgentFold	ContextFold	Ours
Gemini-3-Flash	27.4	32.7	29.8	33.0	36.8
Qwen3.5-35B-A3B	20.3	30.0	27.4	32.0	34.7

a core benchmark in the WebWatcher evaluation [8]; we adopt its released 330 question visual subset and apply the same evaluation protocol as our main experiments (§4.1). Table 13 reports accuracy averaged over three independent runs on two representative backbones. MM-ContextFold achieves the highest accuracy on both backbones, outperforming the ReAct baseline by +4.1 percentage points on Gemini-3-Flash and +4.7 percentage points on Qwen3.5-35B-A3B. These results are consistent with the main findings in Table 1: MM-ContextFold provides a consistent advantage over context-management baselines, including on benchmarks not included in our primary evaluation suite.

D.2 Sensitivity to Branch Budget $B_{\max}$

The branch budget $B_{\max}$ controls the maximum number of steps allowed within a single branch before the agent must return its findings to the main state. We set $B_{\max} = 5$ in all main experiments (§4.1). To assess the sensitivity of this choice, we vary $B_{\max} \in \{2, 3, 4, 5, 6\}$ on Qwen3.5-35B-A3B. For efficiency, we randomly sample 100 examples from each of the seven benchmarks (700 samples in total) and report the average accuracy over this subset; the absolute values are therefore not directly comparable with the full-benchmark results in Table 1.

As shown in Table 14, accuracy increases steadily from $B_{\max}=2$ to $B_{\max}=5$, with the largest jump between 2 and 3 (+0.9 percentage points). Performance peaks at $B_{\max}=5$ (58.7%) and drops slightly at $B_{\max}=6$ (58.2%). The results suggest that a branch budget of 5 provides a favorable trade-off: shorter branches ($B_{\max} \leq 3$) may force premature returns before the agent has gathered sufficient evidence, while longer branches ($B_{\max}=6$) offer diminishing returns and may accumulate redundant tool calls within a single subtask.

Table 14: Sensitivity of MM-ContextFold to the branch budget $B_{\max}$ (average accuracy (%) on the 700-sample subset, Qwen3.5-35B-A3B). All other hyperparameters are held fixed.

	$B_{\max}=2$	$B_{\max}=3$	$B_{\max}=4$	$B_{\max}=5$	$B_{\max}=6$
Qwen3.5-35B-A3B	56.9	57.8	58.4	58.7	58.2

Overall, accuracy varies within a 1.8 percentage-point range across the five settings, indicating that MM-ContextFold is not highly sensitive to this hyperparameter.

D.3 Trajectory Examples

A common MM-ContextFold trajectory follows four stages: (i) the initialization cycle produces coarse image descriptions that help the main state name candidate entities, source types, or visible keywords; (ii) the first image-grounding branch allocates one or a few relevant images and uses *Visual Search* to identify a source page, entity, or clue that cannot be reliably inferred from text alone; (iii) subsequent branches become increasingly text-dominant, using *Text Retrieval* and *Web Visit* to verify the initial hypothesis, collect factual evidence, and reconcile conflicting clues; (iv) the main state synthesizes the folded textual evidence and outputs the final answer without carrying raw visual tokens throughout the trajectory.

We present three correctly solved examples from the Qwen3.5-35B-A3B backbone, spanning multiple benchmark categories, to illustrate how the framework adapts to varying visual demands and task complexity.

D.3.1 Case 1: Identifying a Runway Collection from a Street-Style Photo (LiveVQA). **Question.** “Which organization’s runway collection included the ensemble worn by the woman shown walking on the sidewalk, considering the creative director’s debut was marked by another label that season?” **Reference answer:** Shushu/Tong. The query image shows a woman on a sidewalk wearing a navy-blue two-piece ensemble with a white-collar crop top and a matching midi skirt with white angular detail.

- **Branch 1** (image: <image_0>). Initialization: produces a planner-friendly description of the image, noting the outfit details and street-style setting.
- **Branch 2** (image: <image_0>). Subtask: “Identify the woman and the specific fashion outfit. Find which designer/brand this ensemble is from.” The branch performs a reverse-image search, identifies the outfit as Shushu/Tong, and returns the brand name with a candidate collection context.
- **Branch 3** (image: <image_0>). Subtask: “Confirm the outfit details. Find information about Shushu/Tong’s creative director and their debut season.” The branch corroborates the brand attribution via additional image-based search and surfaces the designers’ names (Liushu Lei and Yutong Jiang).
- **Branch 4** (text only). Subtask: “Search for Shushu/Tong’s creative directors and whether they had a debut at another label that season.” With visual grounding complete, this text-only branch searches for the designers’ career history and confirms the creative-director context referenced in the question.

Output: Shushu/Tong (correct). The transition from image-assigned branches (1–3) to a text-only branch (4) reflects P3: once the brand identity is visually confirmed, the final verification subtask proceeds without pixel-level access. The branches address identification, visual verification, and textual fact-checking.

D.3.2 Case 2: Tracing a University Facility Closure from a Logo (BrowseComp-VL). **Question.** “Which location of this university in the image experienced the earliest permanent closure following a reduction in operating hours?” **Reference answer:** Saxbys coffee shop. The query image shows the University of Pennsylvania logo and branding.

- **Branch 1** (image: <image_0>). Initialization: identifies the image as University of Pennsylvania branding.
- **Branch 2** (image: <image_0>). Subtask: “Confirm UPenn branding via image search; then search for campus locations that have permanently closed.” The branch confirms the university identity and retrieves a list of historical campus relocations, but these are planned relocations, not closures following operating-hour reductions.
- **Branch 3** (text only). Subtask: “Search for UPenn extension centers, satellite facilities, or specialized locations that closed after reducing operating hours.” The branch surfaces several candidate facilities but does not find a definitive match with the “reduced hours then closed” criterion.
- **Branch 4** (text only). Subtask: “Search for any UPenn location that had reduced hours before closing, including medical, continuing-education, or campus retail locations.” This branch discovers the Saxbys coffee shop at 40th and Locust Streets, which eliminated weekend operations in April 2025 and permanently closed on May 2, 2025, after Penn Dental Medicine reclaimed the space.

Output: Saxbys coffee shop (correct). Here the visual-grounding phase is short (a single branch confirms the university identity), after which two text-only branches progressively narrow the search scope (campus relocations → satellite closures → retail closures with hour reductions), illustrating how the main state uses return summaries to refine subsequent subtask prompts.

D.3.3 Case 3: Identifying a Butterfly–Plant Ecological Relationship (VDR-Bench). **Question.** “Which butterfly species in the genus *Eurema* uses the plant as a host, and what is the ecological significance of its nectar for pollinators?” **Reference answer:** *Eurema mexicana* (Mexican Yellow); its nectar supports a wide range of pollinators. The query image shows a plant with fern-like compound leaves and small green buds, alongside a butterfly with iridescent green wings and brown edges.

- **Branch 1** (image: <image_0>). Initialization: describes both organisms—a plant with fern-like compound leaves and a butterfly with iridescent green wings.
- **Branch 2** (image: <image_0>). Subtask: “Identify the plant and butterfly species via image search; then search for which *Eurema* species uses this plant as a host.” The branch identifies the plant as *Acacia angustissima* (Prairie Acacia) and confirms the genus *Eurema* as a candidate host, but the specific species requires further verification.
- **Branch 3** (text only). Subtask: “Search for which *Eurema* species use *Acacia angustissima* as a host plant; find the ecological significance of *Acacia* nectar for pollinators.” No further image access is needed. The branch retrieves documentation confirming *Eurema mexicana* (Mexican Yellow) as the host-associated species and that *Acacia* nectar supports a wide range of pollinators.

Output: *Eurema mexicana* (correct). When the question is relatively focused, MM-ContextFold naturally converges with fewer branches, only one visual branch and one text-only branch beyond initialization. Compared with the four-branch trajectories in Cases 1 and 2, this example shows that the framework does not impose a fixed number of branches but instead lets the main state decide when sufficient evidence has been gathered.

D.4 Illustrative Image-Dependency Cases

We also manually checked a small number of examples from the step-level annotation results used for Figure 2(a) and retained only cases whose original ReAct answers are correct in the corresponding Gemini-3-Flash run. The annotations referenced below are drawn from the two-pass protocol described in §B.4 and Appendix E: the first pass determines *image_needed* (i.e., whether the step extracts new visual information), and the second pass determines *image_info_used* (i.e., whether the step uses any image-derived information at all). These examples help interpret the difference between *Image Needed*, *Image-Derived*, and *Text Only*.

Example 1: *Image Needed* followed by *Image-Derived* (bcv1_level11_12). The question asks how many students at the university shown in the image were commended after placing second in a 2017 tug-of-war competition; the inspected run correctly answers 36. At Step 1, the agent uses visual evidence from the university gate to identify *Harbin Engineering University*, making the step clearly *Image Needed*. At Step 2, the agent performs web search using that identified university name, so the step is no longer directly image-grounded but still relies on information extracted from the image; this is *Image-Derived*. At Step 3, the final synthesis still depends on the university identity recovered from the image, so it remains *Image-Derived* rather than *Text Only*.

Example 2: *Image Needed* followed by genuinely *Text Only* reasoning (bcv1_level11_93). The question asks for the

Hausdorff dimension of the figure described as being constructed by recursively removing the middle third from a line segment; the inspected Gemini-3-Flash run correctly answers *Cantor set*, $\log 2 / \log 3$. The trajectory contains three steps. At Step 1, the agent calls `lens_search` on the query image, which is classified as *Image Needed* because the agent directly submits the image to the visual-search tool. Notably, the image itself depicts a colorful fractal tree, which does not visually resemble the Cantor set described in the question text. At Step 2, the agent recognizes this mismatch, where the textual description “*recursively removing the middle third from a line segment*” already identifies the mathematical object as the Cantor set, and calls `web_search` with a text-based query grounded entirely in the question text rather than any visual detail. The annotator classifies this step as *Text Only* (`image_info_used = false`) because the search query derives from the mathematical description in the question, not from the image content. At Step 3, the agent synthesizes the retrieved mathematical facts and produces the final answer; this step is also *Text Only* for the same reason. This example shows that later non-image steps are not always merely *Image-Derived*: when the decisive evidence is already present in the question text, the trajectory can genuinely switch to a purely textual regime even though the first step used the image.

D.5 Representative Failure Modes

Overly coarse initialization summaries. If the initialization cycle misses a small but decisive clue, the main state may assign an under-specified first branch. This does not invalidate the dual-state design, but it highlights a trade-off: the initialization text should be concise enough for planning while still preserving salient source cues.

Search-engine drift and noisy visual retrieval. Some failures arise when reverse-image retrieval returns near-duplicate distractors, low-quality mirrors, or pages whose titles are only weakly related to the query image. These failures are partly orthogonal to context management: even a better planner cannot recover if the evidence returned by the external search engine is systematically misleading.

Webpage parsing noise. Long or template-heavy webpages can bury the relevant evidence in surrounding boilerplate. In such cases, the branch return may still contain imperfect or incomplete textual evidence even when the correct source page was found.

E Prompts

This section reproduces the full prompt templates used in the final reported runs. We present seven prompts in total: the shared system prompt (§E.1), the three MM-ContextFold state prompts, including main, branch, and initialization (§E.2–4), the answer-evaluation judge prompt (§E.5), and the two-stage grounding-analysis annotator prompts used for the pilot study in Figure 2(a) (§E.6–7).

E.1 System Prompt

All states share a single system prompt that defines the dual-role architecture:

You are a research agent that answers questions by planning and
 ↪ executing focused subtasks. You operate in two states: MAIN (↪ planner) or BRANCH (executor).

E.2 Main-State Prompt

The main-state prompt presents the agent as a research strategist. It requires exactly one action per response: either create a branch subtask (BRANCH) or provide the final answer (FINISH). The `assigned_images` field is the mechanism for adaptive image allocation (P3): it specifies which query images should be loaded into the next branch, and may be an empty list for text-only subtasks. The `image_context` slot holds the image descriptions e_{init} produced by the initialization cycle (§3.3.2).

```

**STATE: MAIN** -- You are the research strategist.

## Workflow
1. Analyze the question and image descriptions
2. Identify what information is still missing
3. Create a focused subtask to gather it
4. Review subtask results, then create another subtask or answer
5. Keep a global round budget in mind; when budget is low, request
  ↪ a synthesis-focused subtask or provide the best calibrated
  ↪ answer.

## Creating a Subtask
Output ONE subtask per response. The executor (BRANCH) will inherit
  ↪ your full context and can use web search, page reading, and
  ↪ reverse image search.

<subtask>
{"description": "3-5 word summary",
 "prompt": "Clear task prompt with objectives. State what to search
  ↪ for, what specific information to extract, and what to return
  ↪ .",
 "assigned_images": ["<image_0>"]}
</subtask>

**`assigned_images` guide:**
- Only assign images that the subtask **directly needs** (e.g., for
  ↪ reverse image search, reading text/numbers, identifying visual
  ↪ details).
- For pure text-based research (e.g., searching a name or fact),
  ↪ use an empty list [] -- no images needed.
- Each image is only loaded inside the BRANCH that receives it,
  ↪ keeping context focused.

## Answering
When evidence is sufficient:
<answer>your final answer</answer>

## Image Context
{image_context}

## Question
{question}

```

Continuation prompt. After each branch returns, the main state receives the following continuation prompt, which presents the branch findings and requires exactly one follow-up action:

```

## Subtask [{description}] completed

{return_message}

---
Review the findings above. Then take **exactly one** action:

**Option A -- Create another subtask:**
<subtask>
{"description": "...", "prompt": "...", "assigned_images": [...]}

```

```
</subtask>
```

```
**Option B -- Provide your final answer:**
<answer>your final answer</answer>
```

Force-answer prompt. When the global budget $T_{\max}$ is reached without a final answer, the last main-state call uses the following prompt:

```
Maximum research rounds reached. You MUST provide your best answer
↪ now based on all evidence gathered.
<answer>your final answer</answer>
```

E.3 Branch-State Prompt

The branch-state prompt presents the agent as a focused executor that inherits the current main-state text and optionally receives a subset of raw query images. In addition to the basic execution loop, the prompt includes convergence-focused rules: a per-branch budget of 5 tool calls, an evidence-validation policy requiring at least two independent signals, and anti-loop and source-quality heuristics.

```
**STATE: BRANCH** -- Execute the assigned research task.

## Available Tools
<tools>
{"name": "search",
 "description": "Perform a Google web search.",
 "parameters": {"query": "string", "goal": "string"}}
{"name": "visit",
 "description": "Visit a URL to extract detailed content.",
 "parameters": {"url": "string", "goal": "string"}}
{"name": "lens_search",
 "description": "Perform Google Lens reverse image search.",
 "parameters": {"image_id": "string", "goal": "string"}}
{"name": "return",
 "description": "Report findings back to MAIN.",
 "parameters": {"message": "string"}}
</tools>

## Response Format
Think step-by-step, then call one tool per response:
<tool_call>
{"name": "tool_name", "parameters": {"key": "value"}}
</tool_call>

## Rules
1. Focus exclusively on the assigned task.
2. When done, you must call `return` to report findings back to
   ↪ MAIN.
3. Budget: finish within at most 5 tool calls; if uncertainty
   ↪ remains, call `return` with the best-supported conclusion and
   ↪ explicit uncertainty.
4. Evidence policy: before `return`, try to validate the key claim
   ↪ with at least two independent signals (e.g., lens_search +
   ↪ search, or search + visit).
5. Anti-loop: do not repeat near-duplicate queries more than twice;
   ↪ if results are noisy, pivot strategy, then conclude.
6. Source quality: prefer official or domain-relevant sources over
   ↪ forums/aggregators when evidence conflicts.

## Task
{task_prompt}
```

E.4 Initialization Prompt

The initialization prompt instantiates the initialization subtask s_{init} (§3.3.2). It is intentionally short and task-agnostic, and is applied to each query image loaded in the initialization branch. Its purpose is

to produce a planner-friendly description d_k of each query image, not to answer the question; the resulting descriptions form e_{init} and are folded into the main context as the image context.

```
Briefly describe this image for a research planner who cannot see
↪ it.
```

1. What the image shows (subject, type of content)
2. Any visible text or numbers
3. What type of source it appears to be from

Output as JSON:

```
{"summary": "...", "source_hint": "...}"
```

E.5 Judge Prompt

The answer evaluator follows the two-stage extract-then-grade protocol of the WebWatcher and Qwen-DeepResearch evaluation template [8, 25], as described in §4.1 and Appendix B. The full prompt is reproduced below.

```
You are grading a predicted answer against a gold target.
```

Task -- two steps:

1. EXTRACT: From the predicted answer, extract the core factual
 ↪ claim that directly answers the question. Write it as a short
 ↪ phrase.
2. GRADE: Compare the extracted claim with the gold target and
 ↪ assign exactly one grade.

Grading rules:

- CORRECT: The extracted claim is semantically equivalent to the
 ↪ gold target. Ignore casing, punctuation, phrasing, order, or
 ↪ minor typos. Hedging is acceptable if the correct answer is
 ↪ clearly stated.
- INCORRECT: The extracted claim contains a factual statement that
 ↪ contradicts the gold target (even if hedged). If correct and
 ↪ incorrect claims coexist, grade INCORRECT.
- NOT_ATTEMPTED: No clear factual claim is made (e.g. "I don't know
 ↪ "), or only partial information is given with no contradictions.
 ↪

Additional rules:

- For numbers, accept values correct to the last significant figure
 ↪ of the gold target.
- The prediction only needs to address what the question asks, even
 ↪ if the gold target contains extra detail.
- Do not penalize omission of info clearly inferable from the
 ↪ question.

Question: {question}

Gold target: {correct_answer}

Predicted answer: {response}

Reply in exactly this format (2 lines, nothing else):

Extracted: <core answer in a short phrase>

Grade: <A, B, or C>

E.6 Grounding-Analysis Annotator Prompt

The grounding annotator (Stage 1) used in Figure 2(a) receives the question and the step-by-step trajectory in chronological order, then classifies each step with a counterfactual rule: *could this step still be produced if the raw image were removed while all previous textual output were retained?* The full prompt is reproduced below.

```
You are analyzing a multi-step reasoning trajectory from a vision-
↪ language agent. The agent was given a question about an image
↪ and used tools (web_search, lens_search, visit) to find the
↪ answer.
```

```

Your task: For each assistant step below, determine whether the
↳ step requires direct access to the original image to obtain NEW
↳ visual information, or whether it merely reuses image-derived
↳ information that was already textualized in a previous step.

## Classification Labels

**"image_needed" = true**:: This step extracts NEW information
↳ directly from the image that has NOT appeared in any previous
↳ step's text. Examples:
- FIRST mention of what the image depicts
- Using lens_search / reverse image search on the original image
- Noticing a NEW visual detail not previously described
- Re-examining the image to verify or correct a previous
↳ observation

**"image_needed" = false**:: This step does NOT require the original
↳ image. All image-related information it uses was already
↳ converted to text in earlier steps. Examples:
- Searching for an entity identified from the image in a PREVIOUS
↳ step
- Reasoning about facts retrieved from web search results
- Mentioning image content that was already fully described earlier
- The step is purely text-based: processing search results,
↳ synthesizing information, or giving a final answer

## Key Principle
Ask yourself: "If I removed the original image from this step's
↳ input but kept all previous steps' text output, could this step
↳ still produce the same result?" If YES -> image_needed = false.
↳

## Question
{question}

## Steps (in chronological order)
{steps_text}

## Output Format
Return a JSON array with one object per step:
[{"step": 1, "tool": "...", "image_needed": true/false, "reason": "
↳ brief explanation (1 sentence)"}]

```

```

**false**:: This step has NO dependency on image information
↳ whatsoever. Examples:
- The step processes purely factual web results unrelated to the
↳ image content
- The step reasons about dates, numbers, or facts that could have
↳ been in a text-only question
- The step synthesizes information that has no connection to the
↳ original image

## Key Principle
Ask yourself: "If this had been a text-only question (no image at
↳ all), could this step still appear in an equivalent reasoning
↳ trajectory?" If YES -> image_info_used = false.

## Question
{question}

## Steps (in chronological order)
{steps_text}

## Output Format
[{"step": 1, "tool": "...", "image_needed": true/false, "
↳ image_info_used": true/false, "reason": "brief explanation (1
↳ sentence)"}]

For steps where image_needed=true, image_info_used must also be
↳ true.

```

E.7 Image-Info Reuse Annotator Prompt

Stage 2 further refines each step's label by asking whether the step uses *any* image-derived information, even if that information was already textualized in earlier steps. Combined with Stage 1, this produces the three-way decomposition used in Figure 2(a):

- *Image Needed* — image_needed=true.
- *Image-Derived* — image_needed=false, image_info_used=true.
- *Text Only* — image_needed=false, image_info_used=false.

```

Each step has already been labeled with:
image_needed: whether this step requires DIRECT access to the
↳ original image

Your additional task: for each step, determine **image_info_used**
↳ -- whether this step's reasoning or action makes use of ANY
↳ image-derived information, regardless of whether it comes
↳ directly from the original image or was textualized in a
↳ previous step.

## Classification for image_info_used

**true**:: This step uses image information in any form. Examples:
- The step directly examines the original image (image_needed=true
↳ implies this)
- The step's search query or reasoning references something
↳ identified from the image in a previous step
- The step's reasoning mentions visual details that came from the
↳ image

```